ALU Project

Table of Contents

Introduction....4

Design Process....5

Top Level ALU....7

Symbol....7

Test Bench & Test Cases....7

Passing A validation (INSTR = 000)....8

Passing B Validation (INSTR = 001)....9

A AND B Validation (INSTR = 010)....10

A OR B (INSTR = 011)....11

A XOR B (INSTR = 100)....12

A ADD B (INSTR 101)....12

Subtractor (INSTR = 110)....15

All High (INSTR = 111)....18

Full Adder:....19

Transistor level schematic....19

Full Adder Symbol....19

Test Bench & Test Cases....20

4-Bit Adder/Subtractor....22

Block/Gate Level Schematic....22

Symbol....23

Test Bench and Test Cases.... **Error! Bookmark not defined.**

2:1 MUX....24

Transistor Level Schematic....24

Symbol....25

Test Bench & Test cases....25

2:1 Mux with 4-Bit Inputs....27

4-Bit 8:1 MUX....28

Block Level Schematic....28

Symbol....28

Test Bench & Test Cases....29

2-Input AND....31

Transistor Level Schematic....31

Symbol 31
Test Bench & Test Cases 32
2-Input AND (4 Bits) 34
2-Input OR: 36
Transistor Level Schematic 36
Symbol 36
Test Bench & Test Cases 36
2-Input OR (4 bits) 38
2-Input XOR 39
Transistor Level Schematic 39
Symbol 39
Test Bench & Test Cases 40
2-input XOR (4-Bit) 42
Full Adder Layout 43
Schematic 44
Stick Diagram 44
Layout 46
DRC 47

Introduction

In this project report, I show the design process behind the ALU architecture I designed. The purpose of designing this ALU is to showcase the skills developed throughout the course. The ALU is designed and implemented using Cadence Virtuoso in CMOS .18μm technology with 2:1 PMOS and NMOS ratio. Figure 1 shows the block level circuit of the ALU to be designed.

Figure 1: Block Level ALU Circuit

The ALU will have 3 inputs and 2 outputs. The first 2 inputs (A and B) are 4-bit data inputs with a 3-bit instruction opcode input. The instruction input determines the function of the ALU. The outputs of the ALU are a 4-bit output determined by the instruction input and 1-bit overflow to indicate overflow of the 4-bit adder/subtractor. The functionality can be seen in table 1.

Table 1: ALU Truth Table

INSTR<2:0>	OUT<3:0>	Description
000	A	Pass A
001	B	Pass B
010	= (A AND B)	Bitwise AND
011	= (A OR B)	Bitwise OR
100	= (A XOR B)	Bitwise XOR
101	= A + B	Add
110	= A - B	Subtract (using two's complement)
111	= 1111	All high

Design Process

The first step I took when designing my ALU was to first make truth tables for the desired functionality of the ALU. At first glance there are 6 main blocks that make up the functionality of this ALU; those being AND, OR, XOR, Add, subtract and a 8:1 MUX (all of which are 4 bit inputs and outputs). Given this, the first step I took in the design of this ALU was to make the mux, since ultimately this will be the deciding factor behind correct output functionality.

The first truth table I made was for the 2:1 mux since at least 3 components of the block relied on the mux (8:1 mux (4-bit), 2:1mux in logic block, muxes in 4-bit 2:1 adder/subtractor). I designed the mux using based on the 2:1 PMOS and NMOS ratio I determined W for all NMOS to be 1μm and all PMOS to be 2μm based on equations 2 and 3. Based on this design, I designed a 4-bit 2:1 Mux by making 4 separate 2:1 muxes connected 1 bit from A<3:0> and B<3:0> like seen in figure 29. After the 4-bit 2:1 Mux was created I used it to create a 8:1 mux inspired by a 16:1 Mux seen on the GeeksforGeeks website [1]. The selection bits needed for this MUX matched the number of bits the instruction opcode has. I confirmed number of bits needed by using the following equation, where N is the number of inputs,

$$\log_2(N) \quad (1).$$

In my test bench, I simply connected Vpulses to test the combinations of inputs can be and matched them to the ones in the truth table. I did the same thing for the 4-bit 2:1 Mux but only for the A<0> and B<0> since testing one bit would be the same as testing the rest due to the structure of the other bits being identical.

The next step in the design process was to the 4-bit adder/subtractor. I used the same transistor level full adder used in classes and sized the widths of NMOS and PMOS accordingly. Using the following equation is how I determined width size with 2:1 ratio in mind,

$$w_p = N * 2\mu \quad (2)$$

$$w_n = N * 1\mu \quad (3)$$

In equations 2 and 3, N is the number of NMOS or PMOS cascaded together. This same process was used in sizing all transistor level circuits. After the FA was made and turned

into a symbol, I made the same ripple carry adder. To carry out 2s complement subtraction I had the 4-bit 2:1 MUX connected to the inputs of the ripple carry adder. The inputs of the mux were both b<3:0> and b'<3:0>. When the selector bit was high then the inverted inputs would be passed to the 4-bit adder, completing subtraction functions. When selector bit was low, non-inverted inputs would pass completing normal addition functions.

After making the full adder was the creation of the OR, AND, and XOR. Creating the OR and AND was simple, I made a NAND and added inverter transistor circuit to the end to flip the logic to that of an AND. This same process was done for NOR. Making the 4-bit AND and OR was the same as the mux; I attached each respective bit to a single gate resulting in 4-bit logic. The XOR was made by using same structure seen in class (NAND only), I made it using NAND transistor circuits, and it resulted in a single bit XOR. I made 4-input XOR by doing what I did with OR and AND.

Finally, the logic blocks, the logic connected to the adder/substractor uses a 2:1 mux since I will always result in low signal when instructions are 101 and High when instructions are 110. This is shown below in the table below

Table 2: 2:1 MUX Truth Table

INSTR<3> = Sel	INSTR<0>=I_0	INSTR<1>=I_1	Output
0	0	X	L
0	1	X	H
0	0	X	L
0	1	X	H
1	X	0	L
1	X	0	L
1	X	1	H
1	X	1	H

The overflow logic block was implemented directly into the adder subtractor. Overflow during addition occurs when a carry out bit occurs during the addition of the most significant bit (MSB). The reason this is, is because the bit value of the carry out bit is larger than that of the MSB. Overflow in subtraction works a little different, the sign bit in 2's complement is only valid when the carry bit of the MSB is the same as carry out of MSB otherwise the sign bit is not correct due to it not being flipped correct leading to a wrong sign. My overflow logic consists of (Cin(MSB) XOR Cout(MSB)) OR Cout(MSB) to account for both addition and subtraction.

Top Level ALU

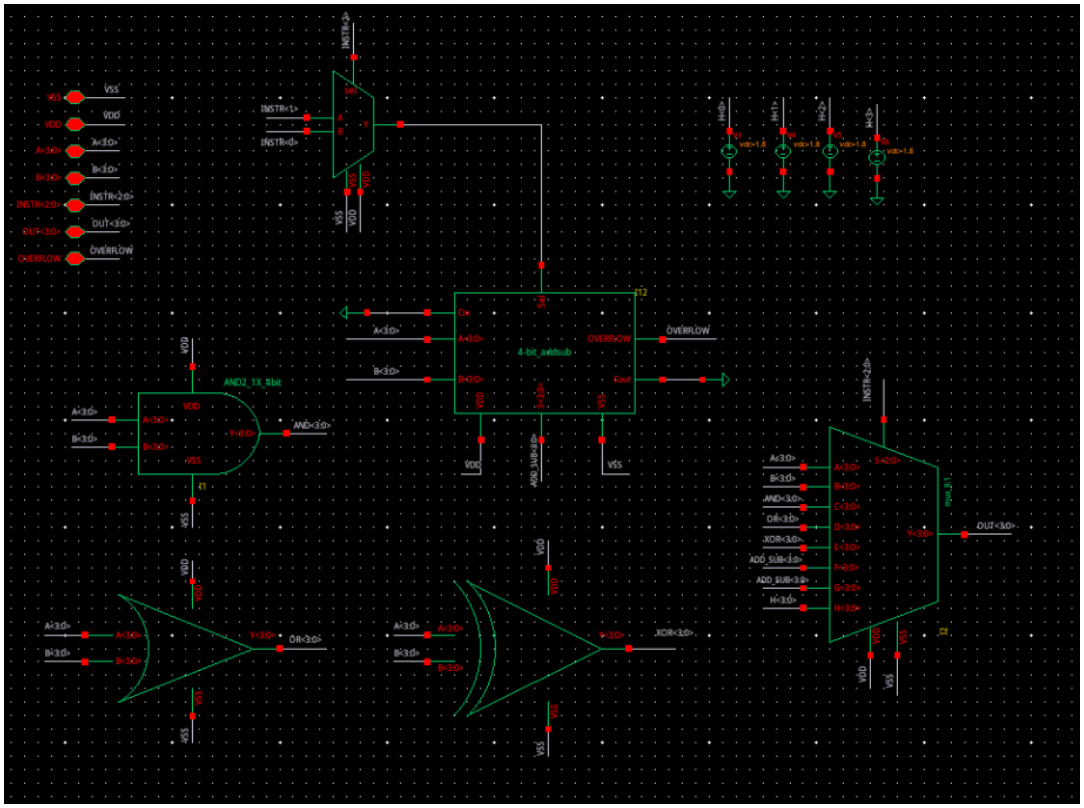

Figure 2: Top Level ALU Architecture

Symbol

Figure 3: ALU Symbol

Test Bench & Test Cases

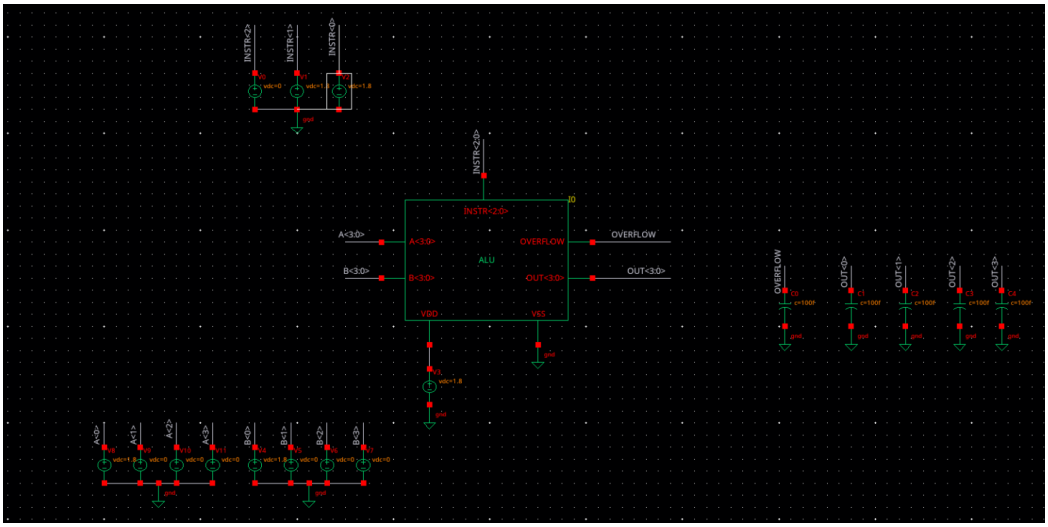

Figure 4: ALU Test Bench

Passing A validation (INSTR = 000)

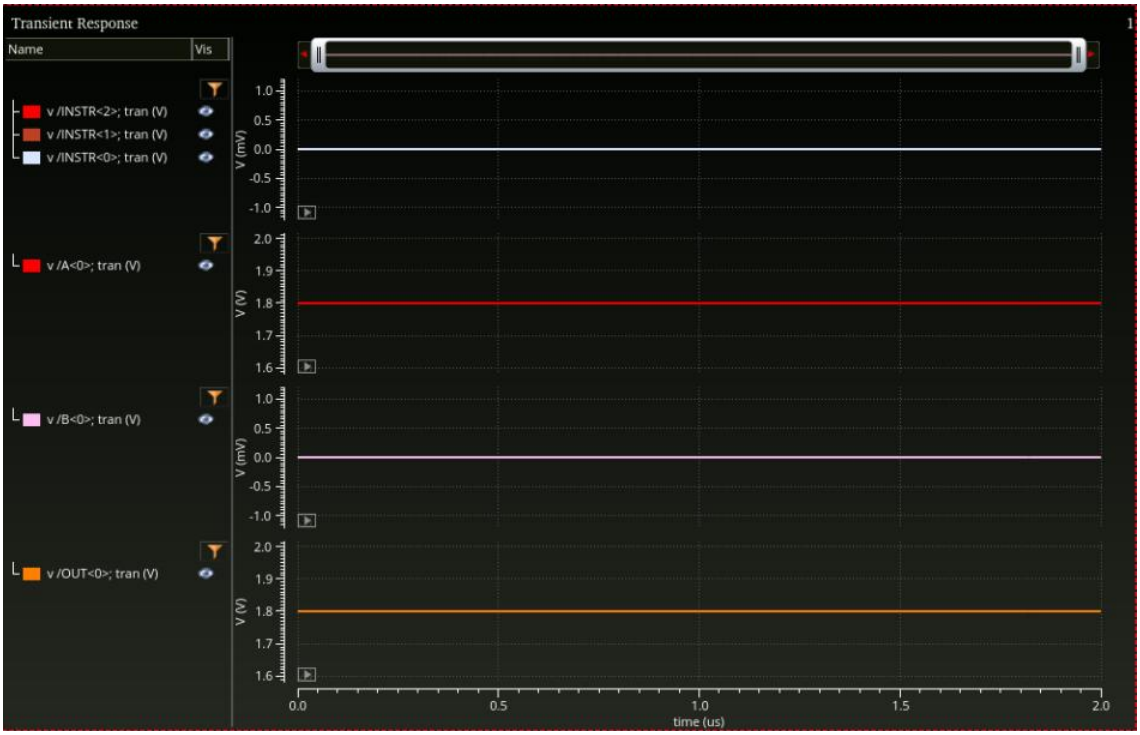

Figure 5: Timing diagram showing OUT = A (A=0001, B=0000)

Passing B Validation (INSTR = 001)

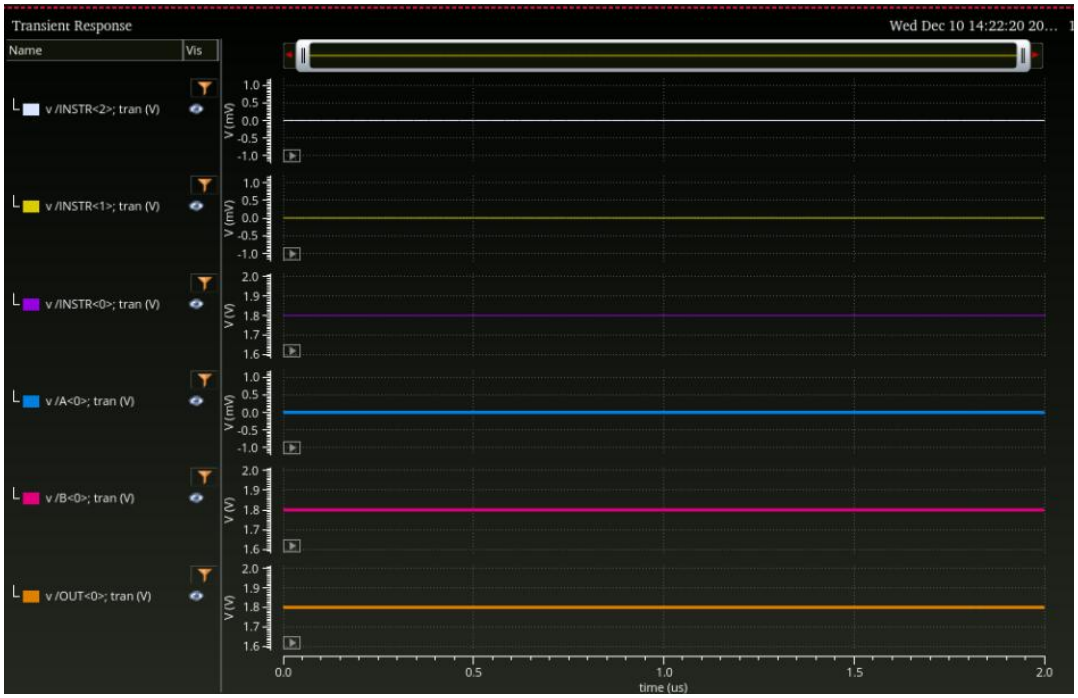

Figure 6: Passing B Timing Diagram (A = 000, B =001)

A AND B Validation (INSTR = 010)

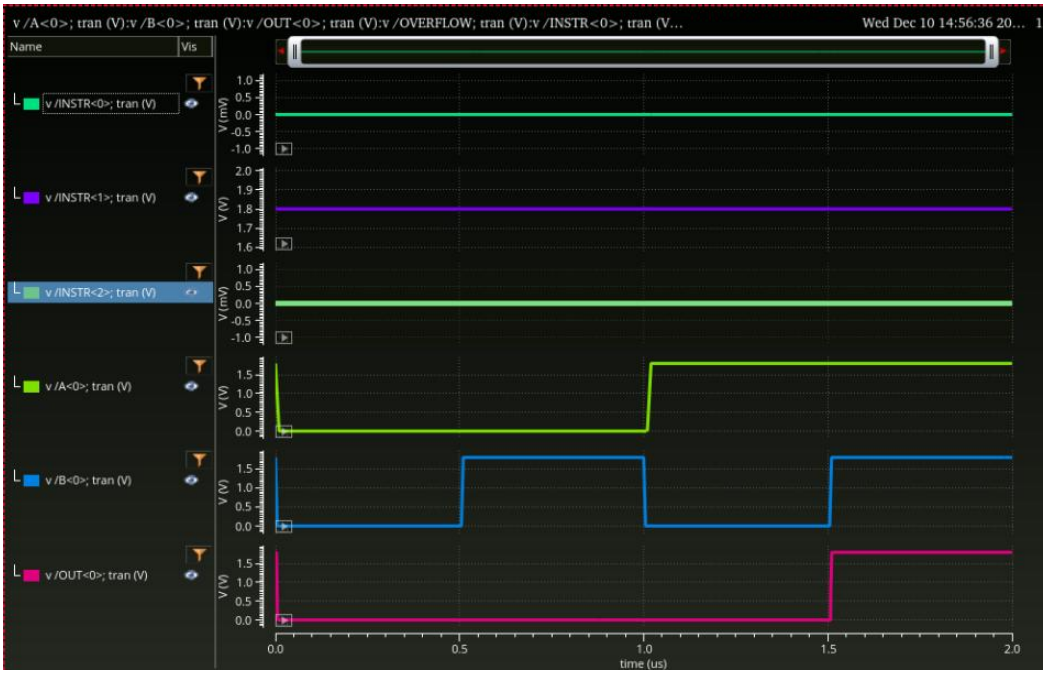

Figure 7: A AND B Timing Diagram

Shows correct functionality as table 7, validating AND functionality

A OR B (INSTR = 011)

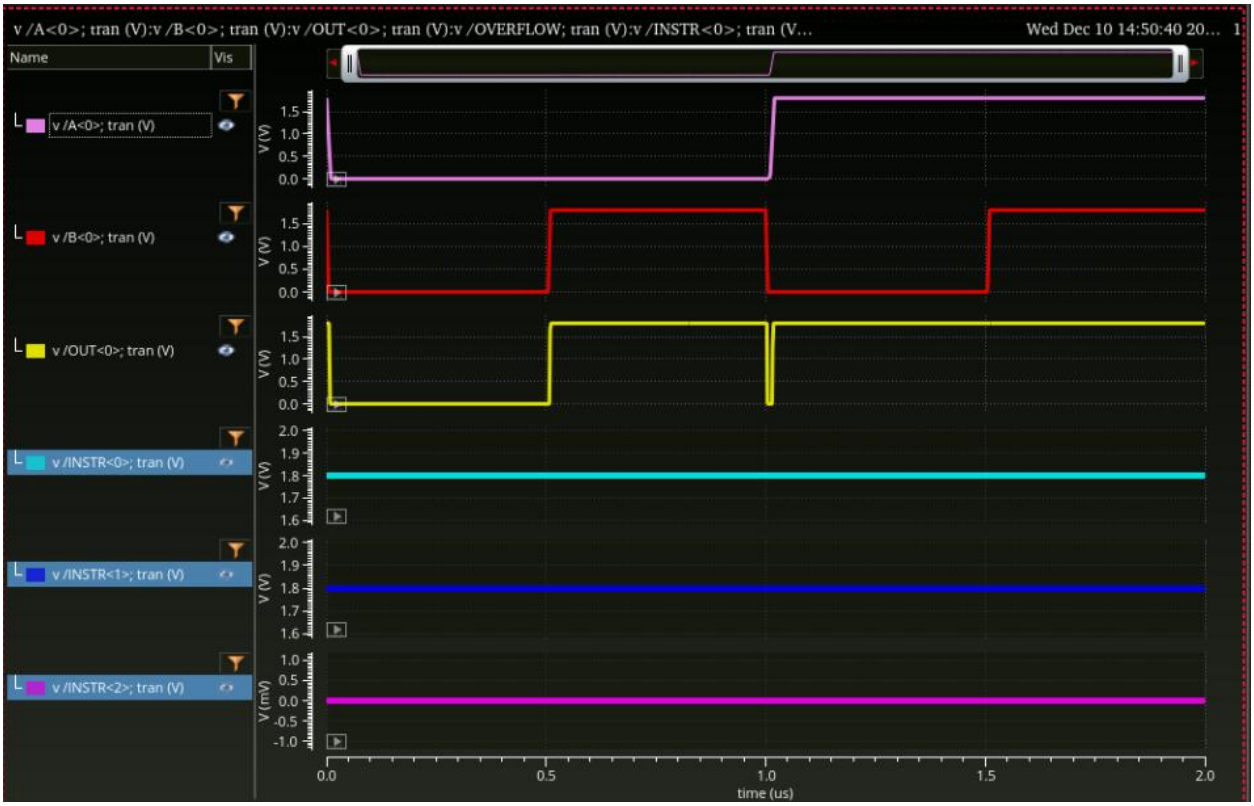

Figure 8: A OR B timing diagram

Shows correct functionality as table 8, validating OR functionality

A XOR B (INSTR = 100)

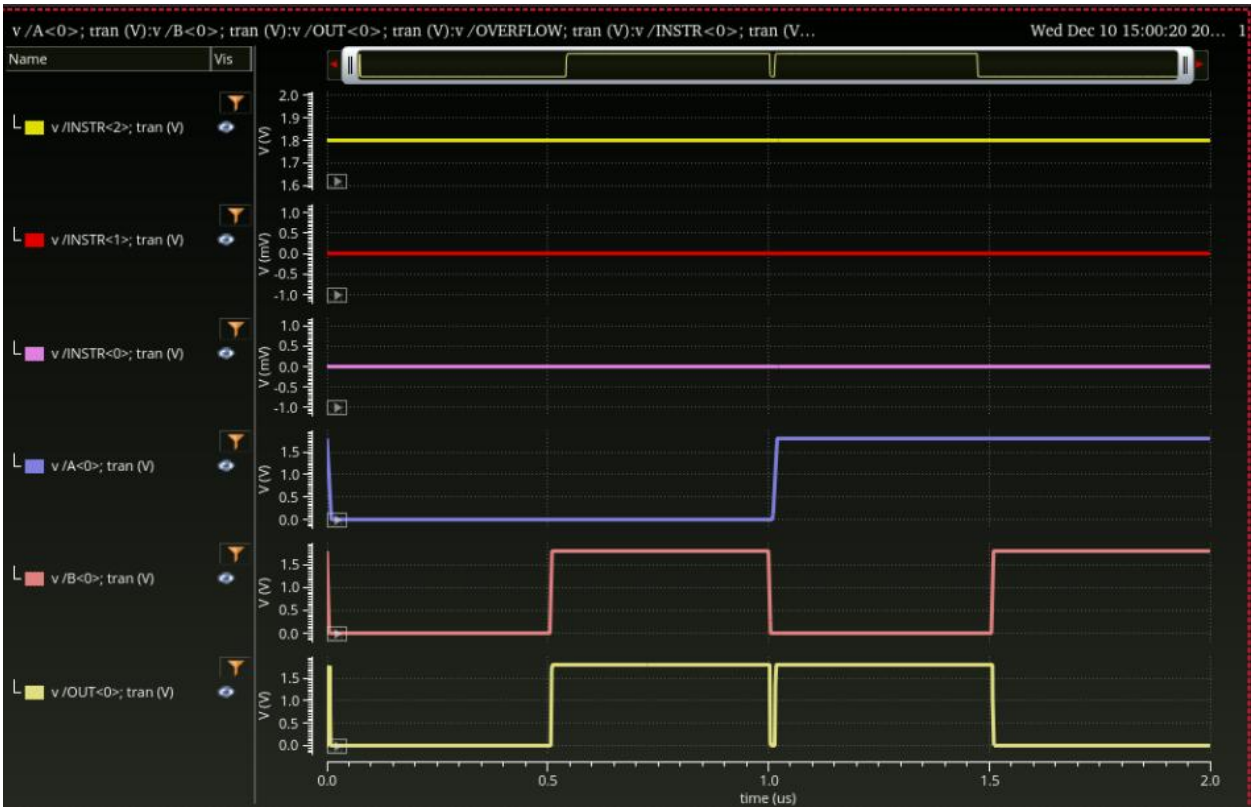

Figure 9: A XOR B Timing Diagram

Shows correct functionality as the table 9, validating XOR functionality

A ADD B (INSTR 101)

Figure 10: A(0001) + B(0000) Timing Diagram

The figure above shows the functionality that adds the least significant bit (LSB). The resulting output of LSB is High. This makes sense since the addition of A (High) and B (Low) should not result in cout being high or sum being low.

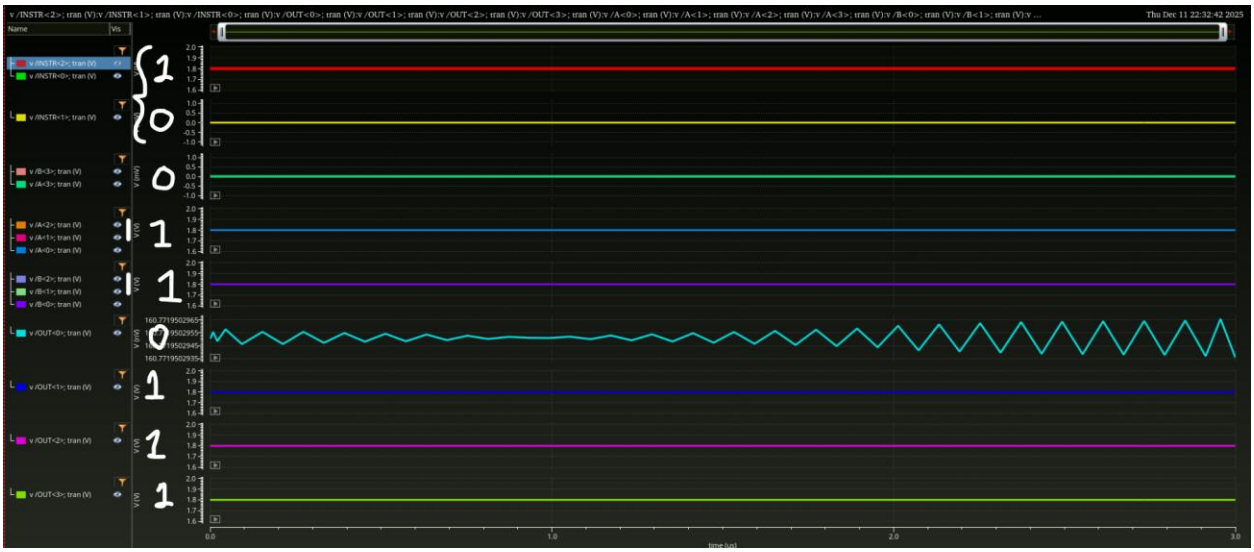

Figure 11: A(0111)+B(0111) Timing Diagram

Figure 11 shows the correct functionality of the adder, since output is 1110 (which is the expected result). This shows correct adder functionality since bit 1 results in a cin to the second bit. When bit 2 adds A and B (both high) resulting in a sum of 1 and cout going to bit 3. This same process happens in bit 3 showing correct functionality of addition in bit 3. The cin going into and the addition of A and B when low results in a sum of 1 at bit 4. Thus the full adder is validated when no over flow is expected.

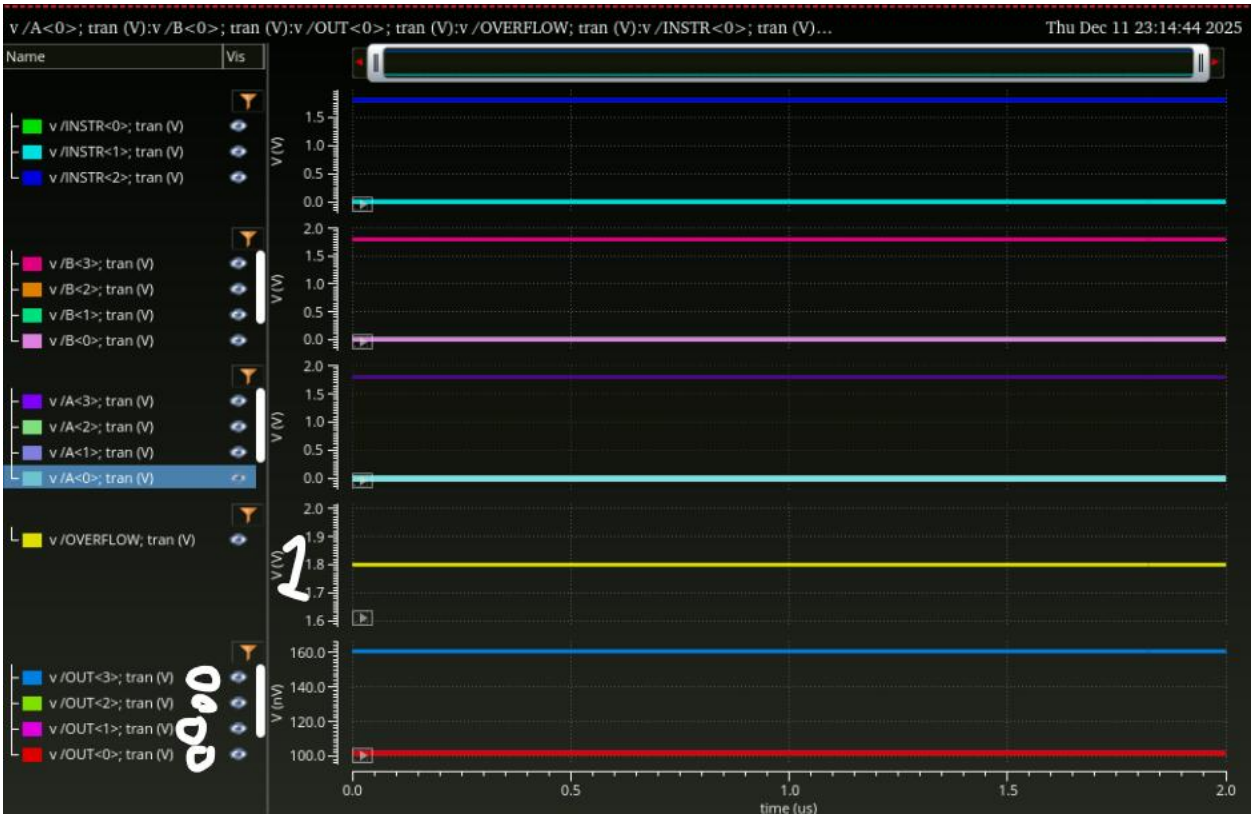

Figure 11: A(1000) + B(1000) Timing Diagram

This timing diagram shows that the functionality of overflow works correctly. When A and B at bit 4 are both high this results in a cout which is the indicator that overflow is occurring in binary addition since this would mean the output is no longer 4 bits.

Subtractor (INSTR = 110)

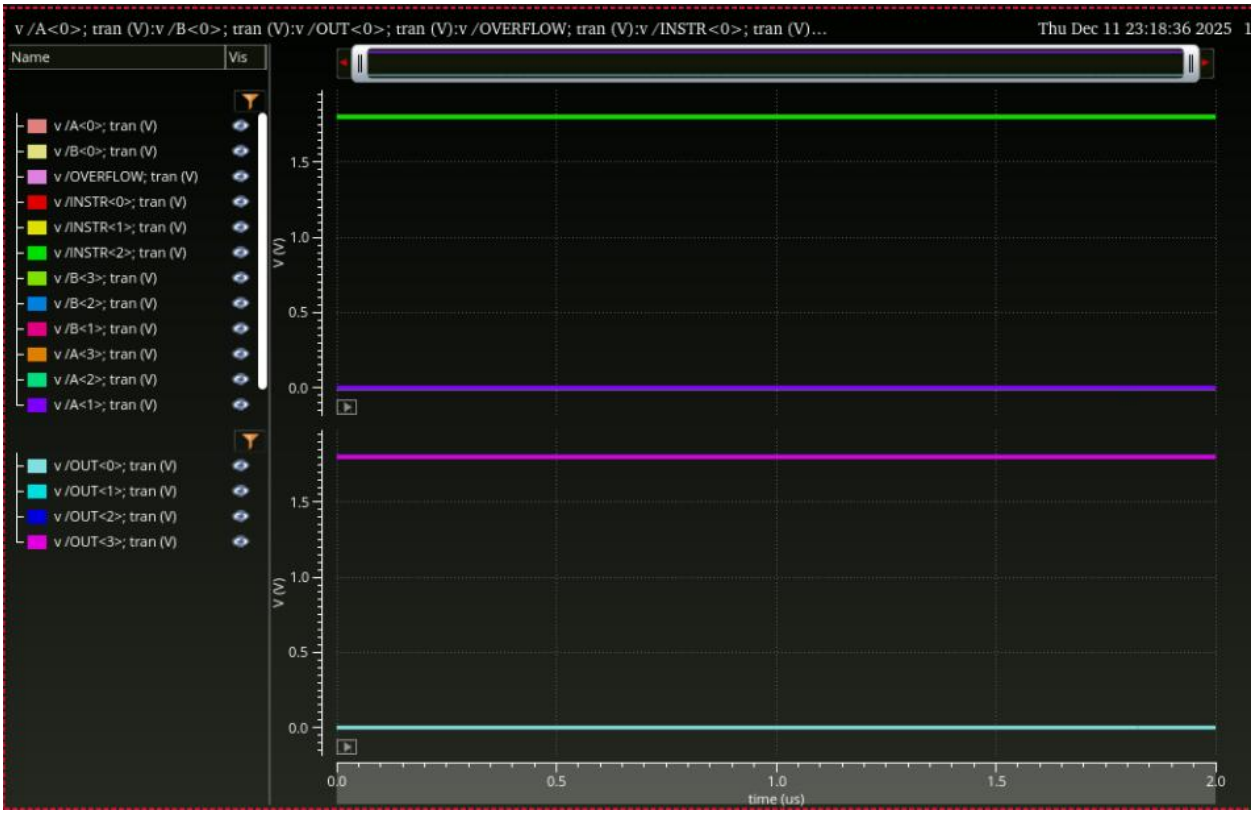

Figure 12: A(0000) – B(0001) Timing Diagram

Shows subtraction borrowing resulting in 1110

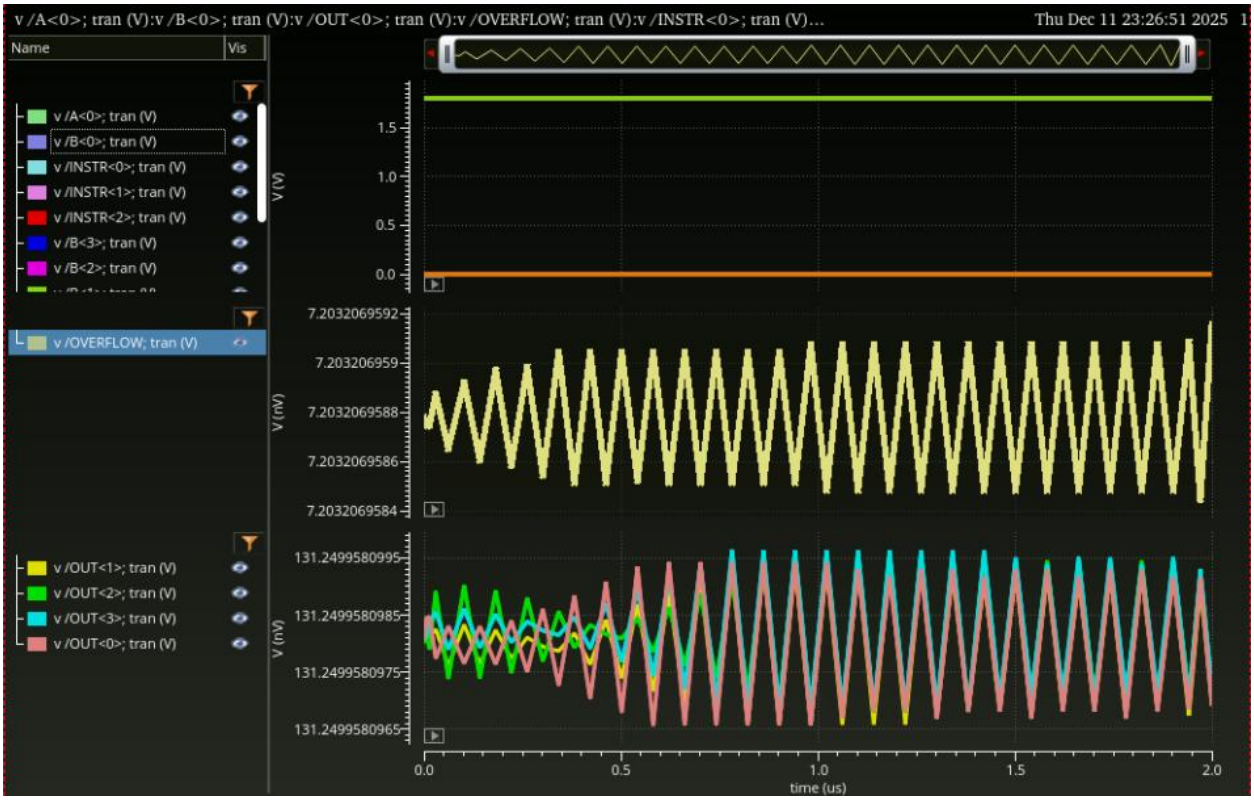

Figure 13: A(0000) – B(1111) Timing Diagram

Shows borrowing across all bits

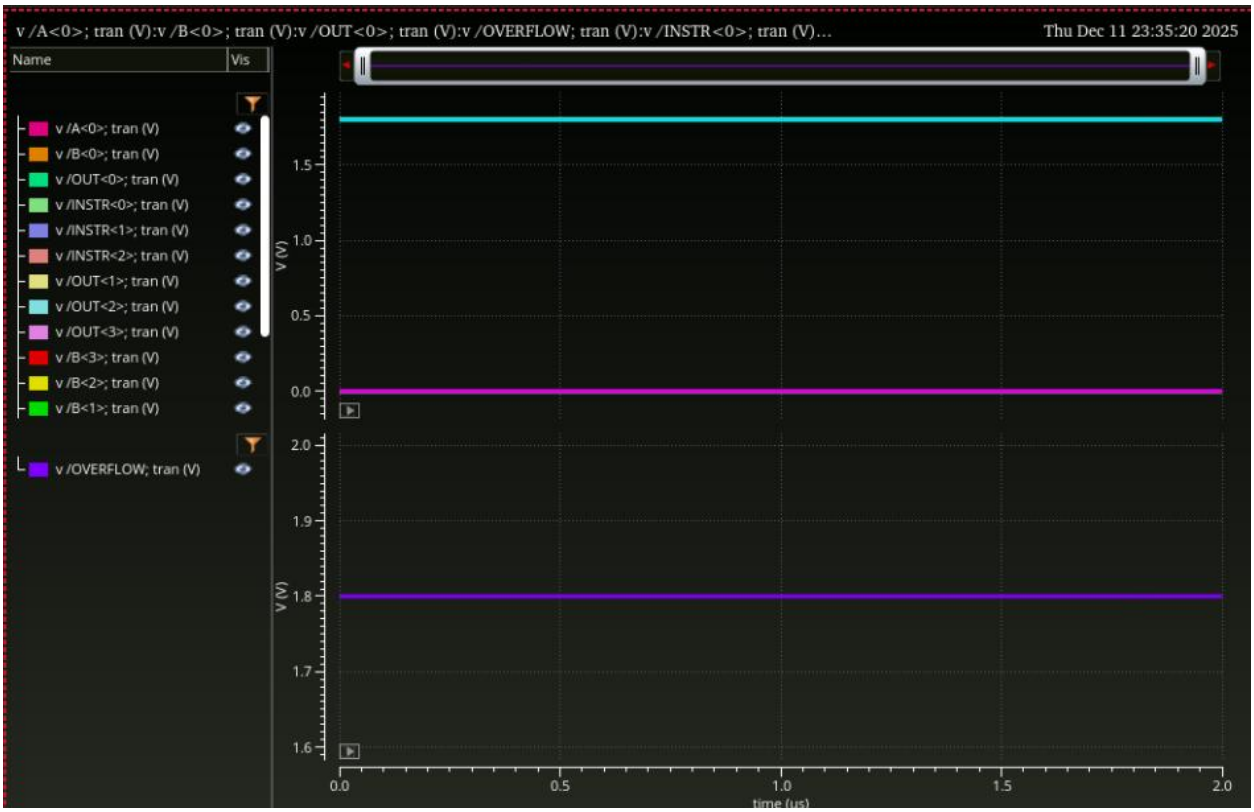

Figure 15: A(1000) – B(0001) Timing Diagram

Results in a negative value that cannot be represented with 4 bit 2s complement subtraction.

All High (INSTR = 111)

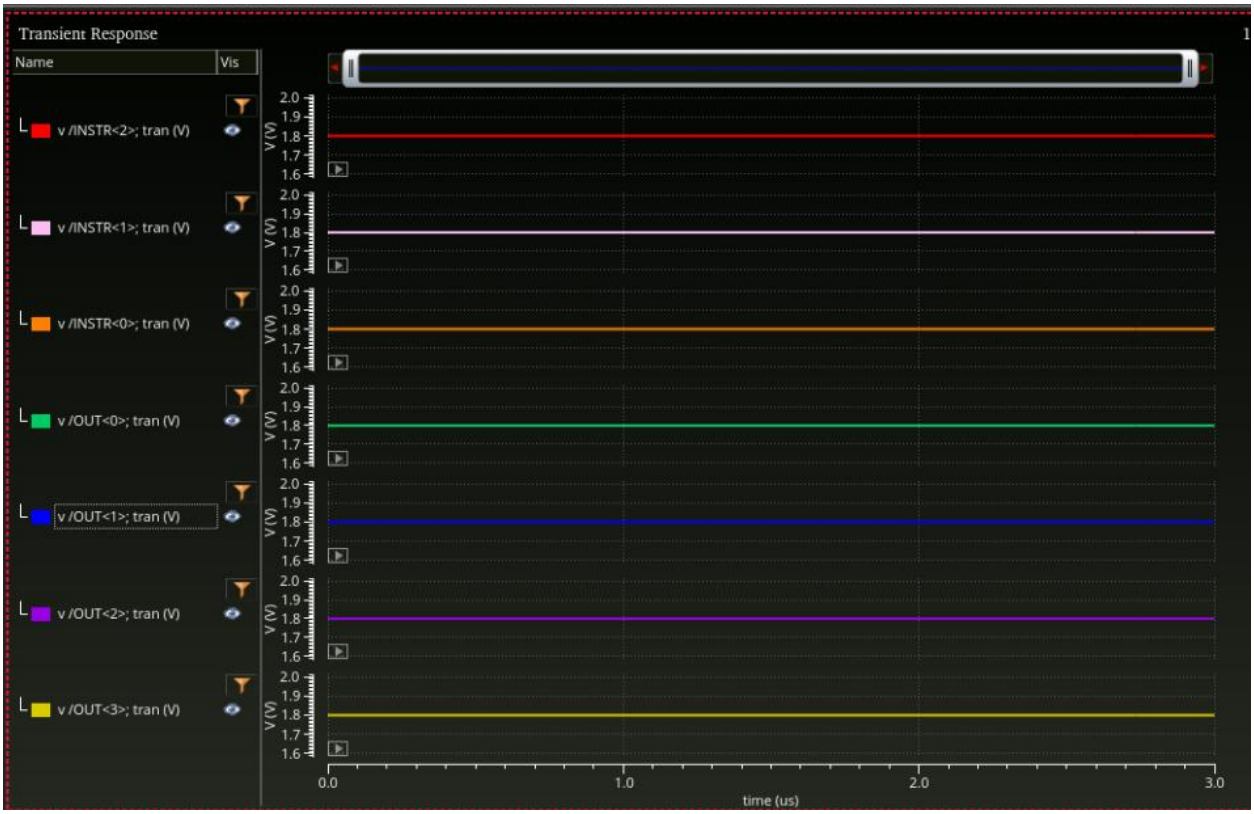

Figure 16: Timing Diagram Showcasing all High Output

When instruction opcode is all high the ALU should output a value that is all high (1111)

Full Adder:

Transistor level schematic

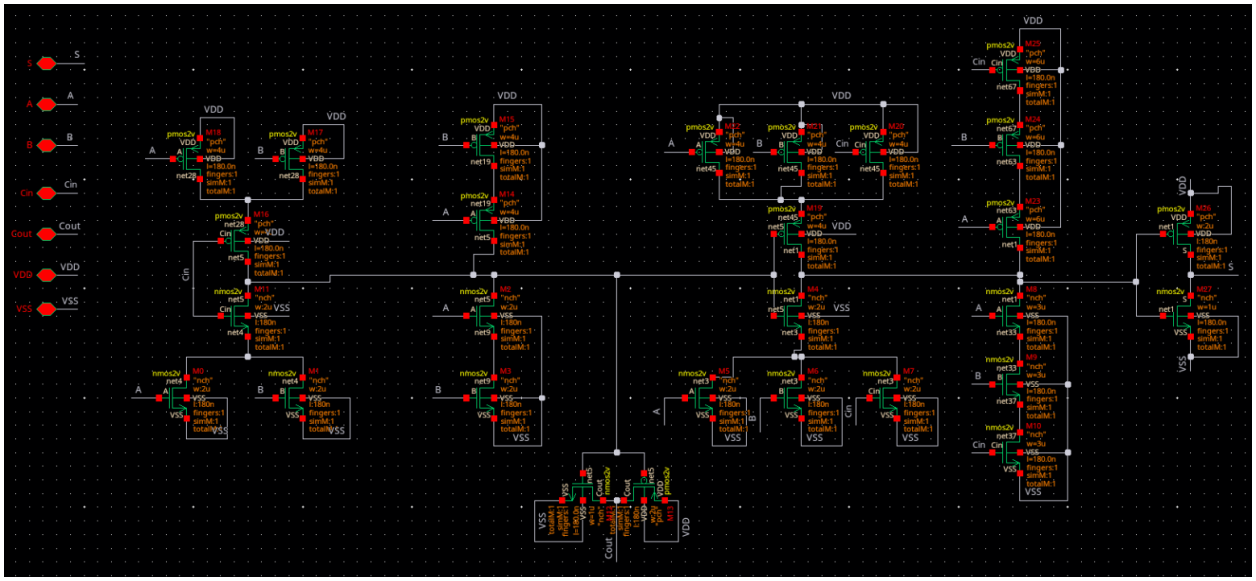

Figure 17: Transistor Level Schematic of the Full Adder

Full Adder Symbol

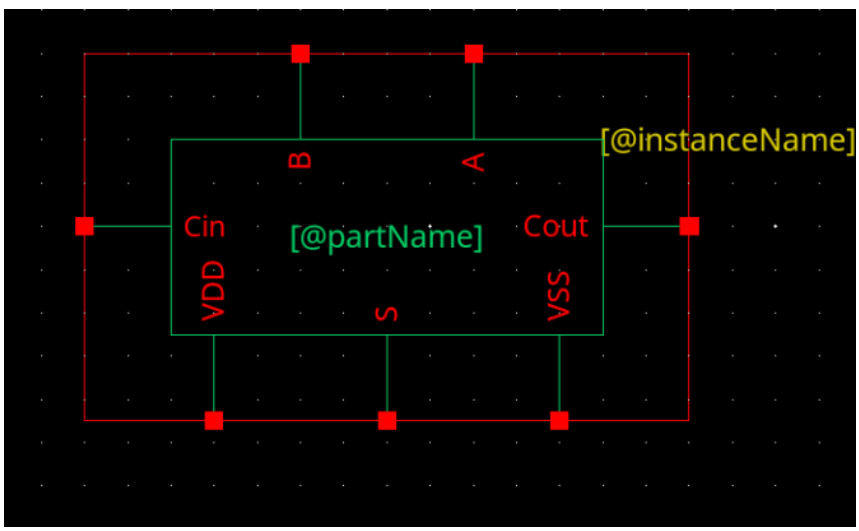

Figure 18: Full Adder Symbol

Test Bench & Test Cases

Table 3: Full Adder Symbol

A	B	Cin	S	OVERFLOW = Cout (only applies to MSB when multiple bits are present)
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

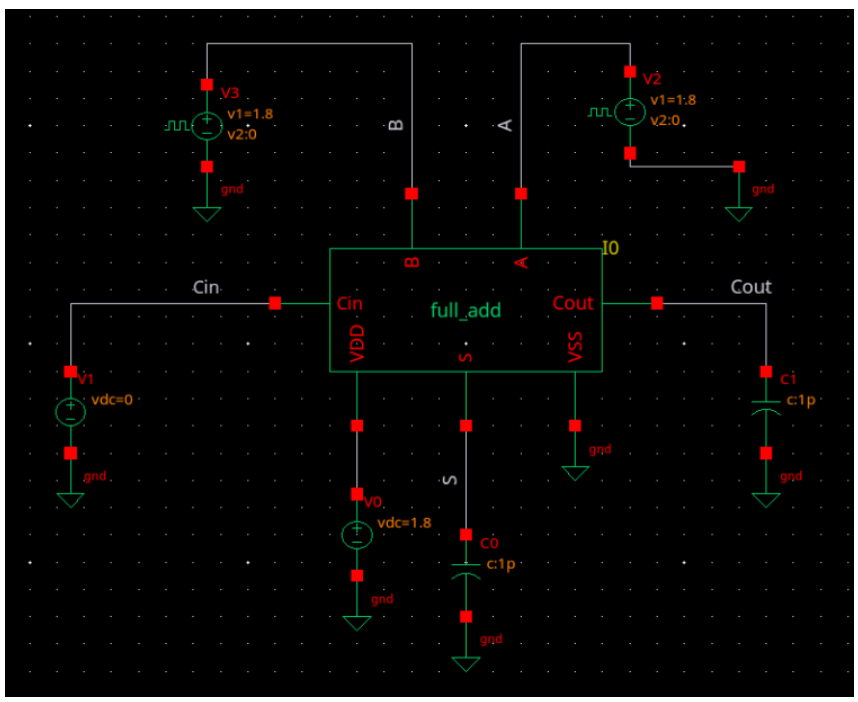

Figure 19: Full Adder Test Bench

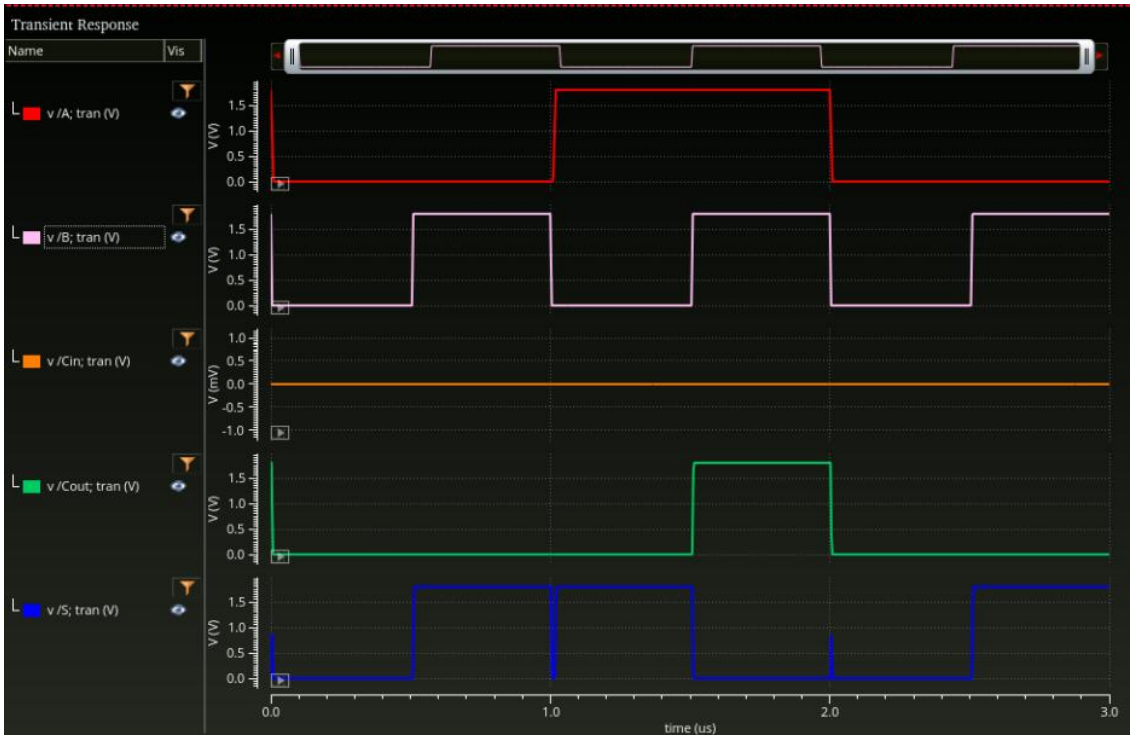

Figure 20: Full Adder Timing diagram (Cin = 0)

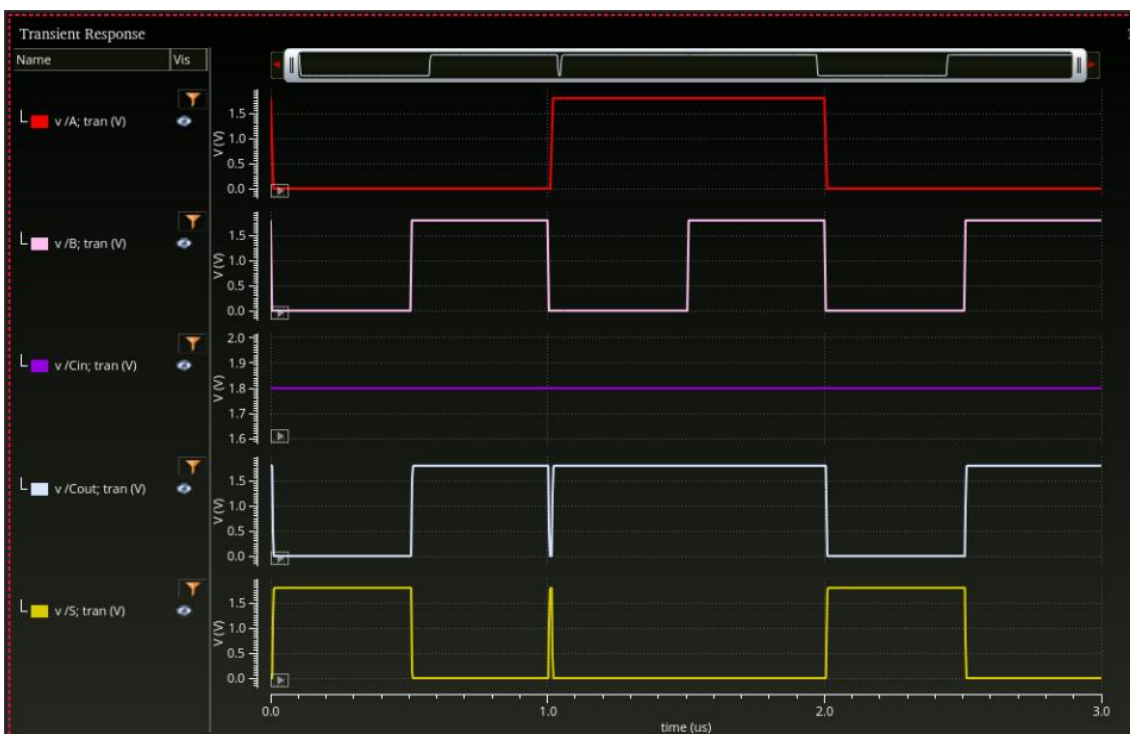

Figure 21: Full Adder Timing diagram (Cin = 1)

4-Bit Adder/Subtractor

Block/Gate Level Schematic

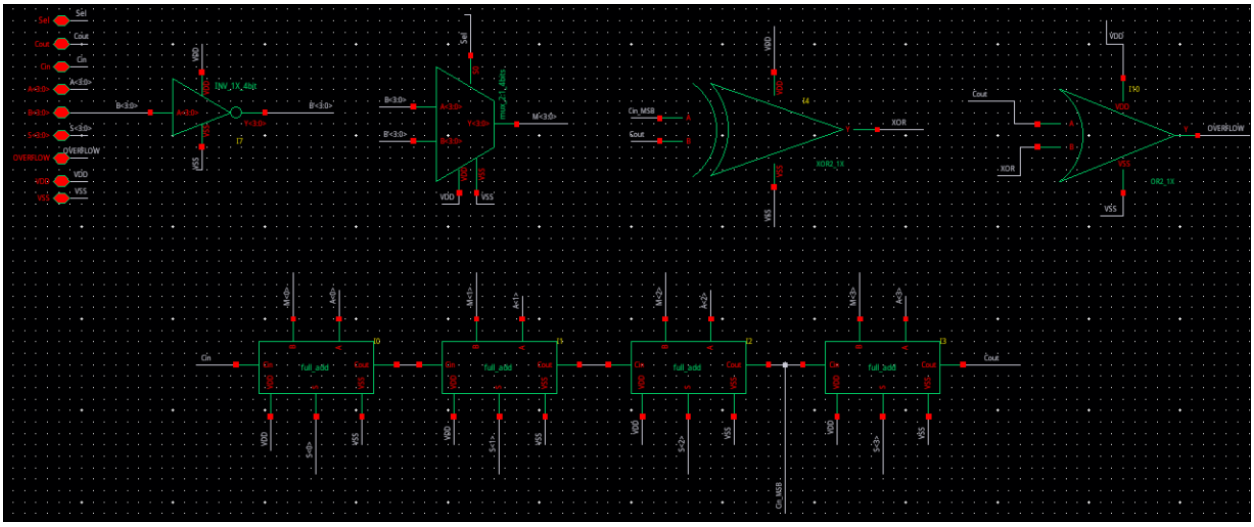

Figure 22: 4-Bit Adder/Subtractor Block/Gate Level Schematic

Symbol

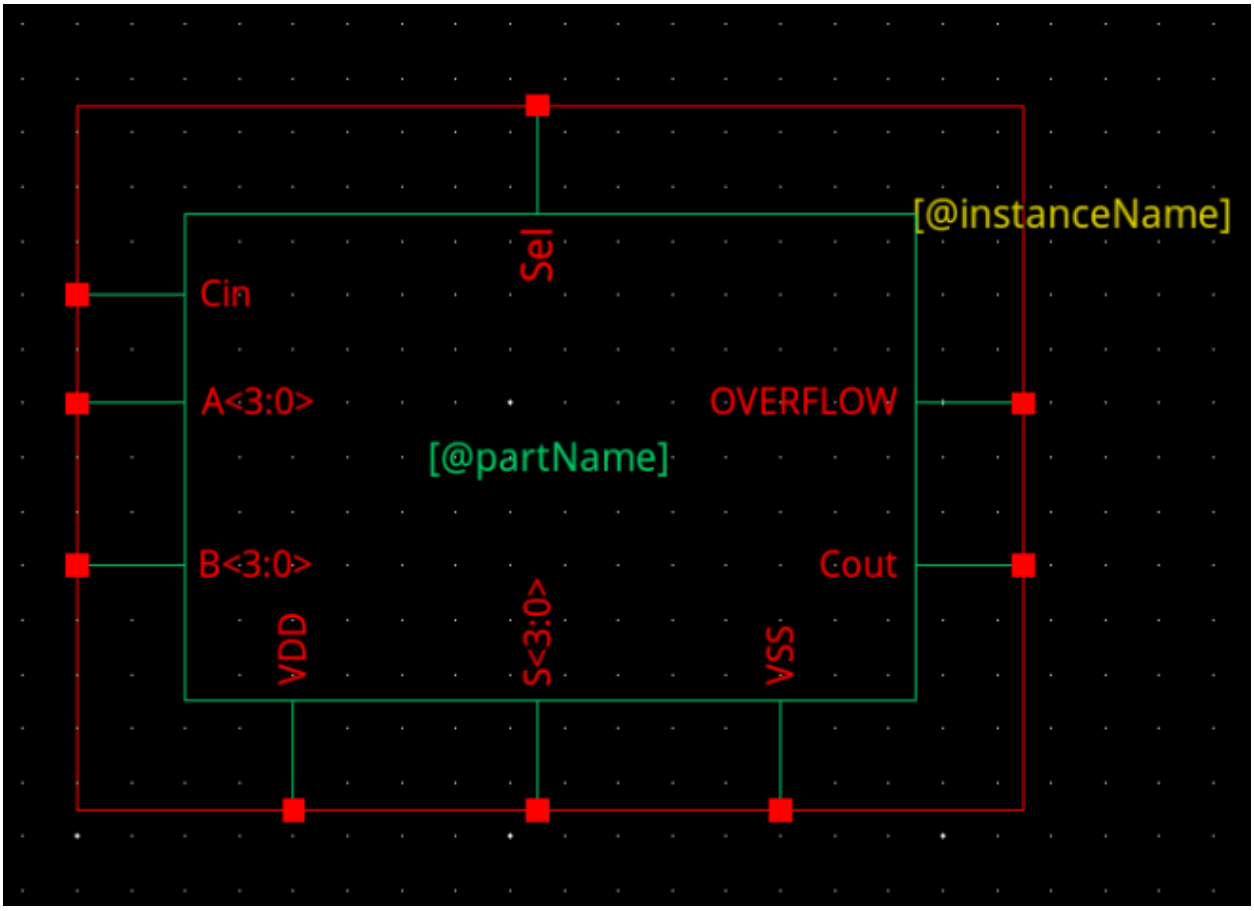

Figure 23: 4-Bit Adder/Subtractor Symbol

Table 4: Full Subtractor (2's Complement) Symbol

A	B	Cin	S	Cout	OVERFLOW = Cout XOR Cin (only applies to MSB when multiple bits are present)
0	0	0	0	0	0
0	1	0	1	1	1
1	0	0	1	0	0
1	1	0	0	0	0
0	0	1	1	1	0
0	1	1	0	1	0
1	0	1	0	0	1
1	1	1	1	1	0

Functionality has been validated above in the ALU testbench.

2:1 MUX

Transistor Level Schematic

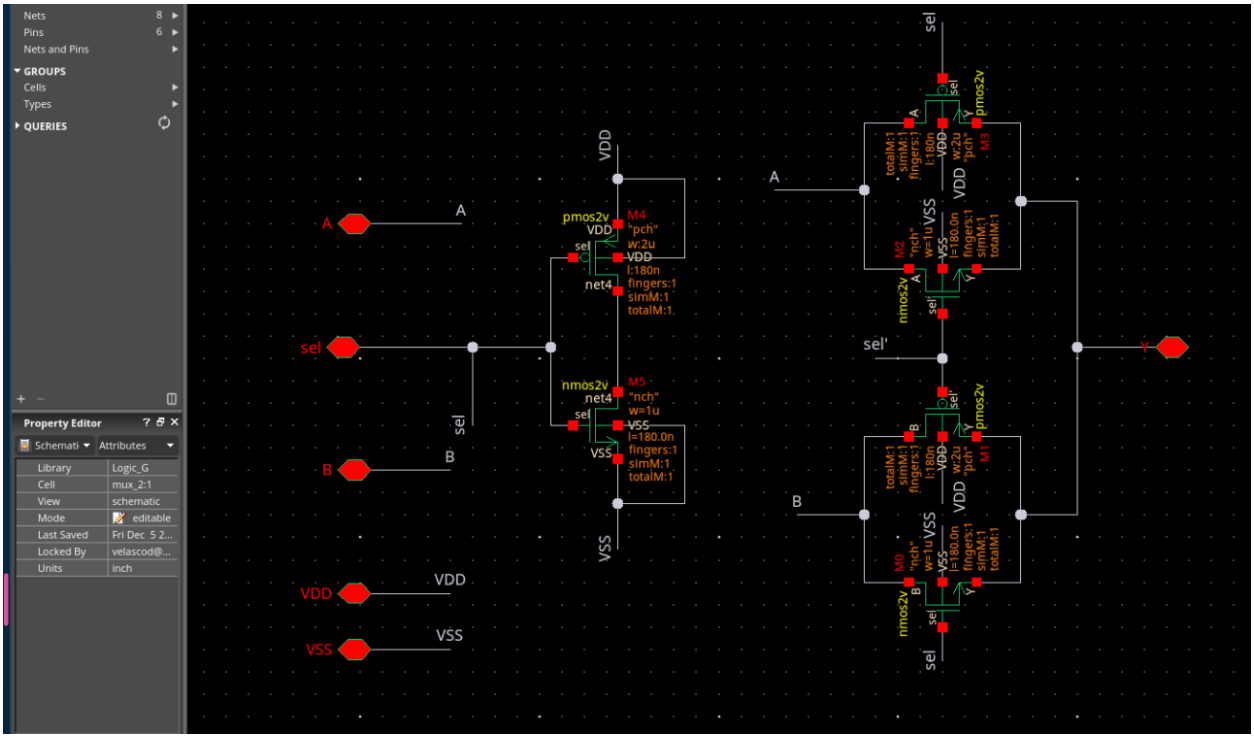

Figure 24: 2:1 MUX Transistor Level schematic

Symbol

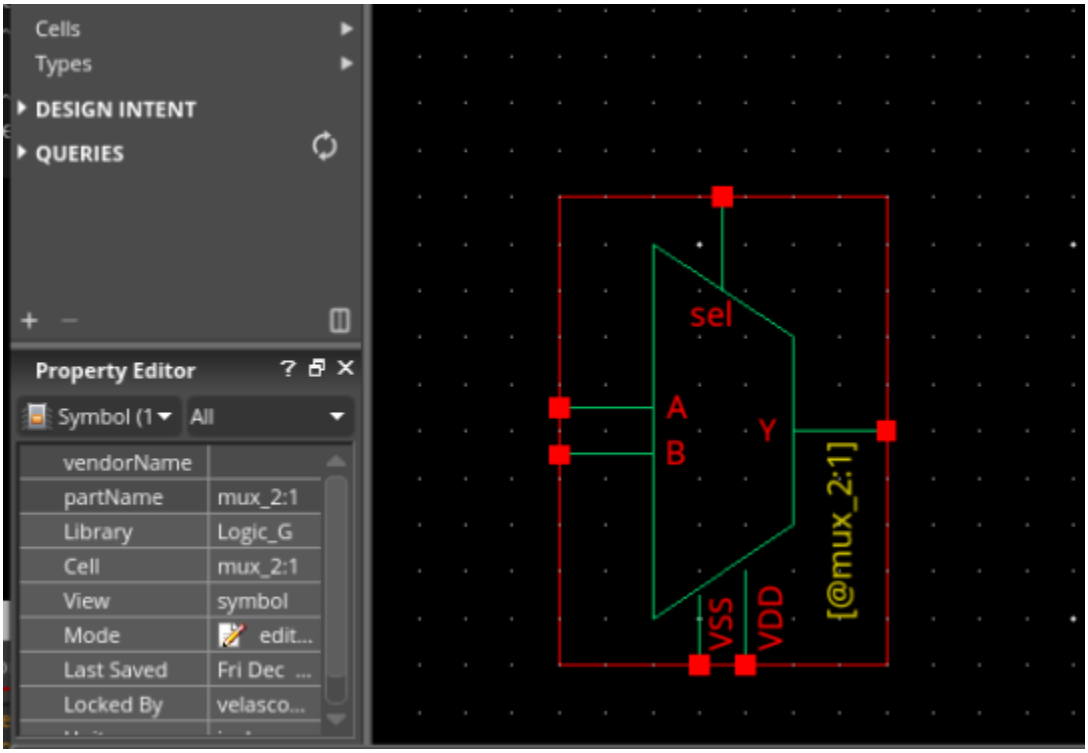

Figure 25: 2:1 MUX Symbol

Test Bench & Test cases

Table 5: 2:1 MUX Truth Table

Selector	A	B	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

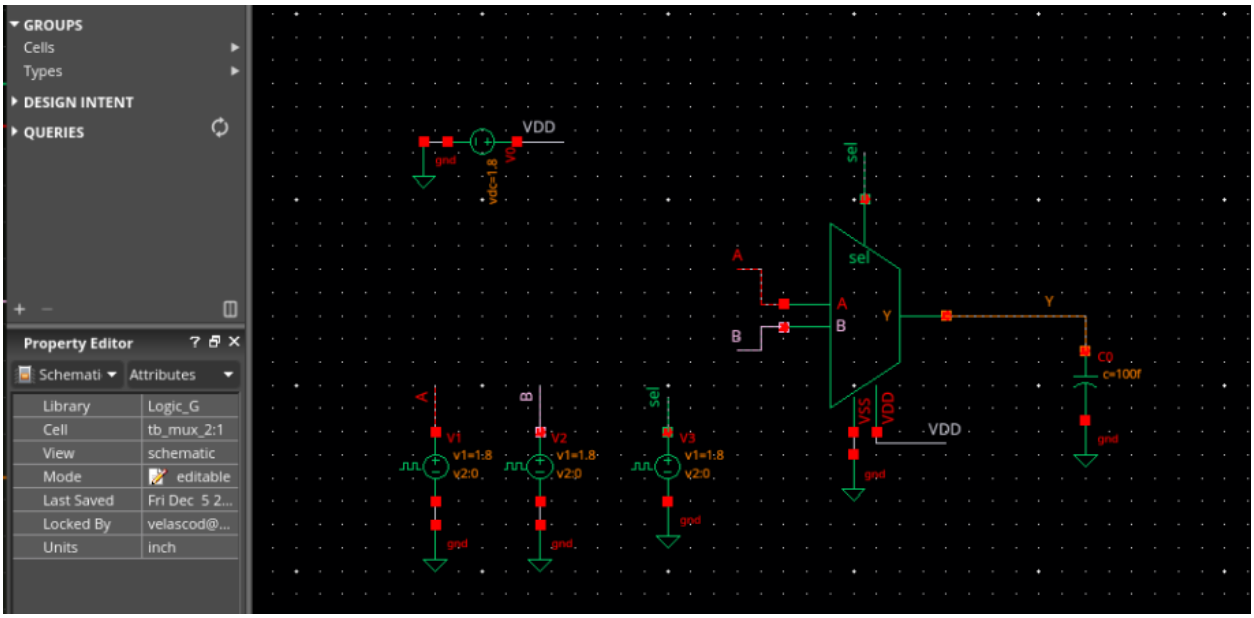

Figure 26: 2:1 MUX Test Bench Schematic

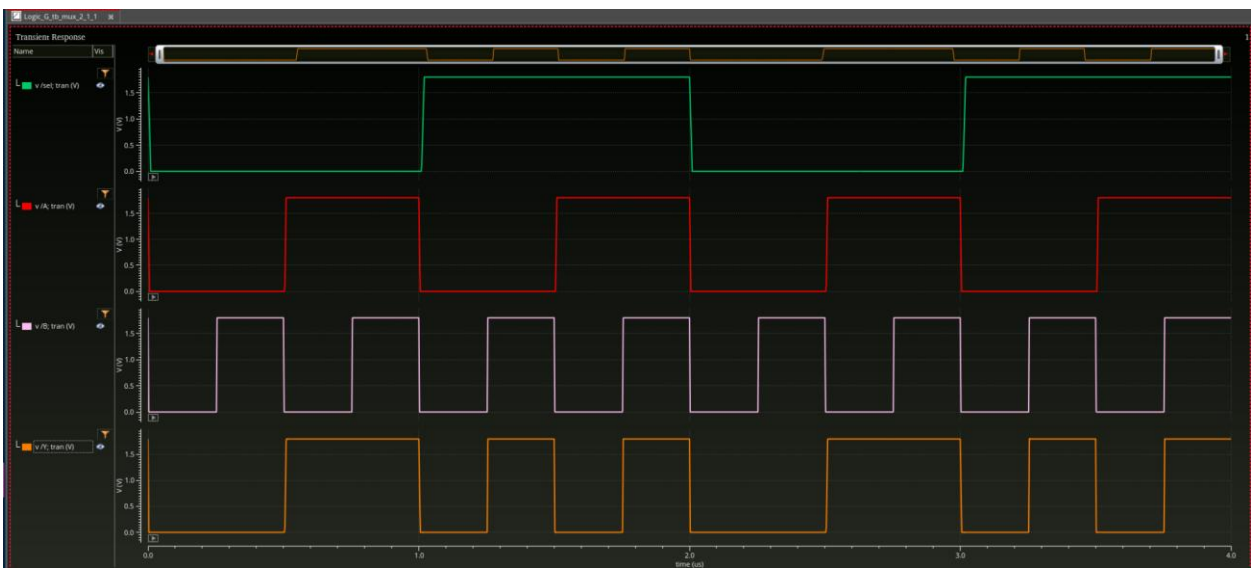

Figure 27: 2:1 MUX Timing Diagram

2:1 Mux with 4-Bit Inputs

Block Level Schematic

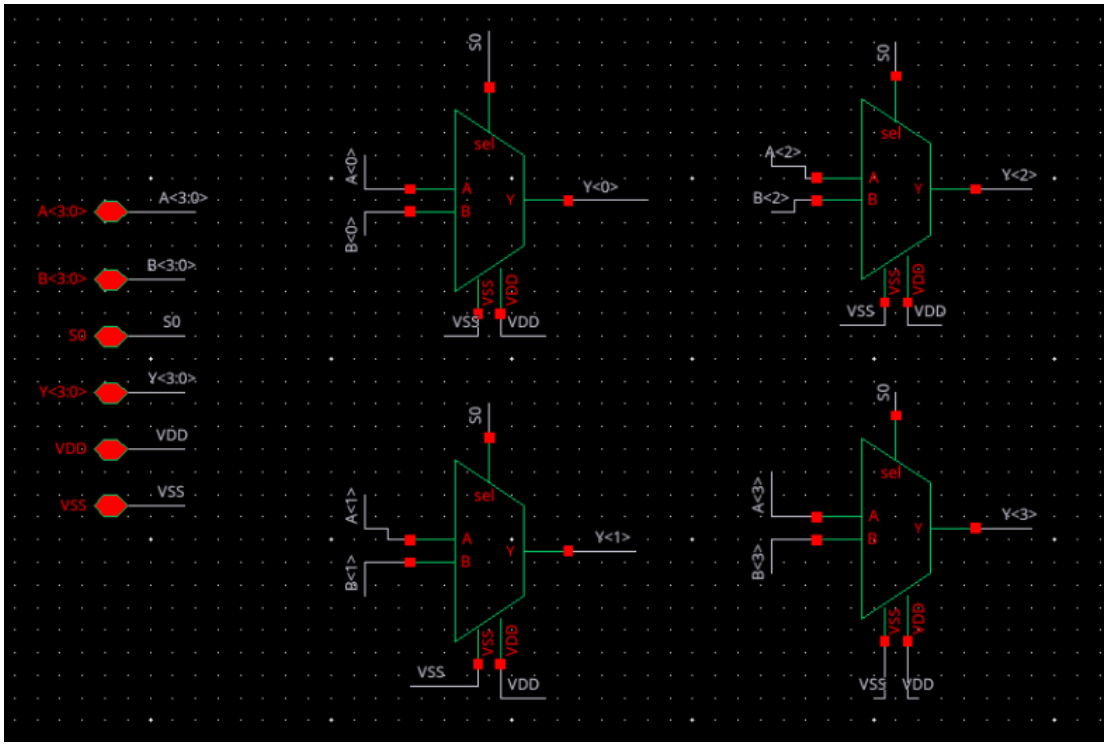

Figure 28: 4-Bit 2:1 MUX Block Level Schematic

Symbol

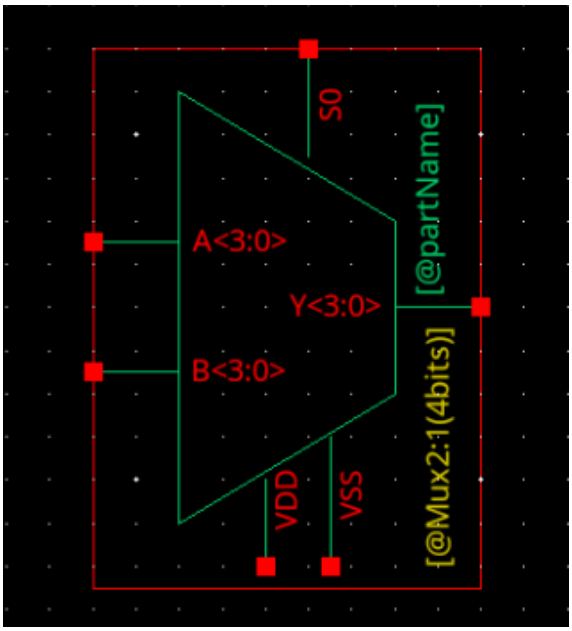

Figure 29: 4-Bit 2:1 MUX Symbol

4-Bit 8:1 MUX

Block Level Schematic

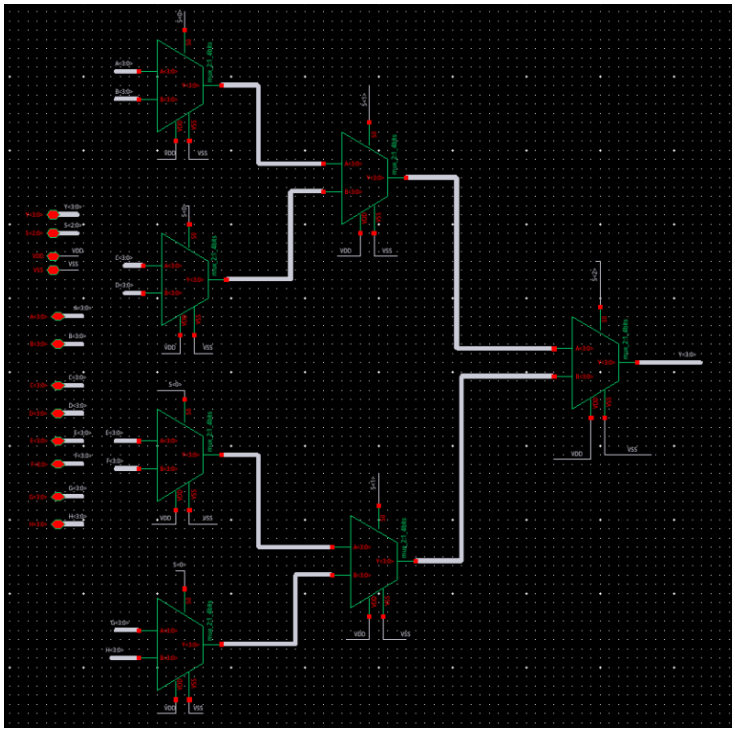

Figure 30: 4-Bit 8:1 MUX Block Level Schematic

Symbol

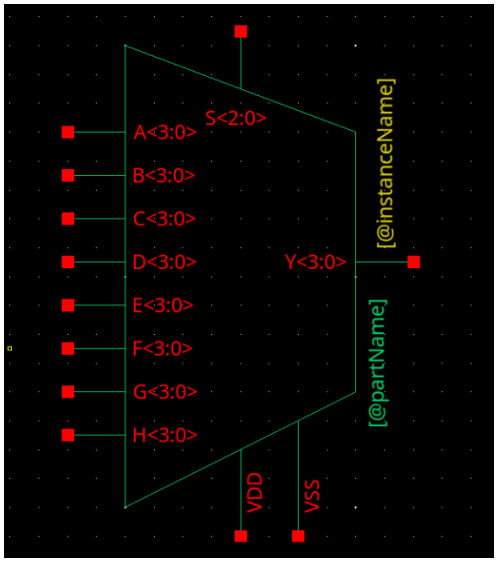

Figure 31: 4-Bit 8:1 MUX Symbol

Test Bench & Test Cases

Table 6: 4-Bit 8:1 MUX Truth Table

S2	S1	S0	Y
0	0	0	A
0	0	1	B
0	1	0	C
0	1	1	D
1	0	0	E
1	0	1	F
1	1	0	G
1	1	1	H

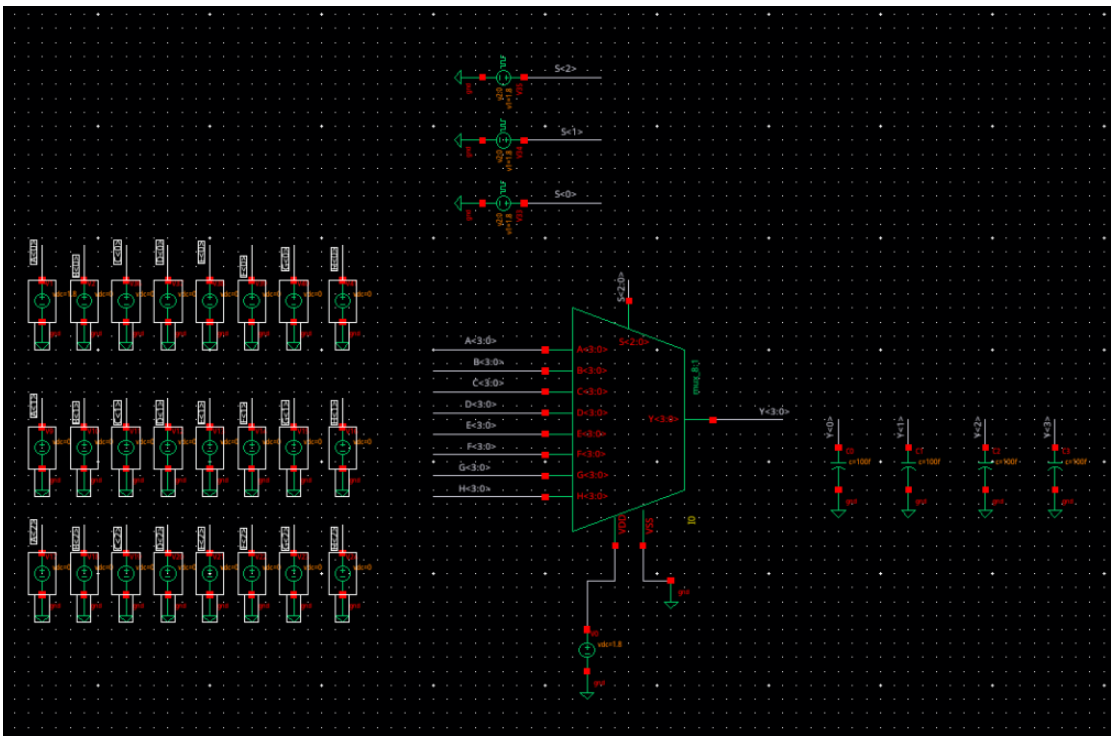

Figure 32: Mux 8:1 (4-bit) Test Bench

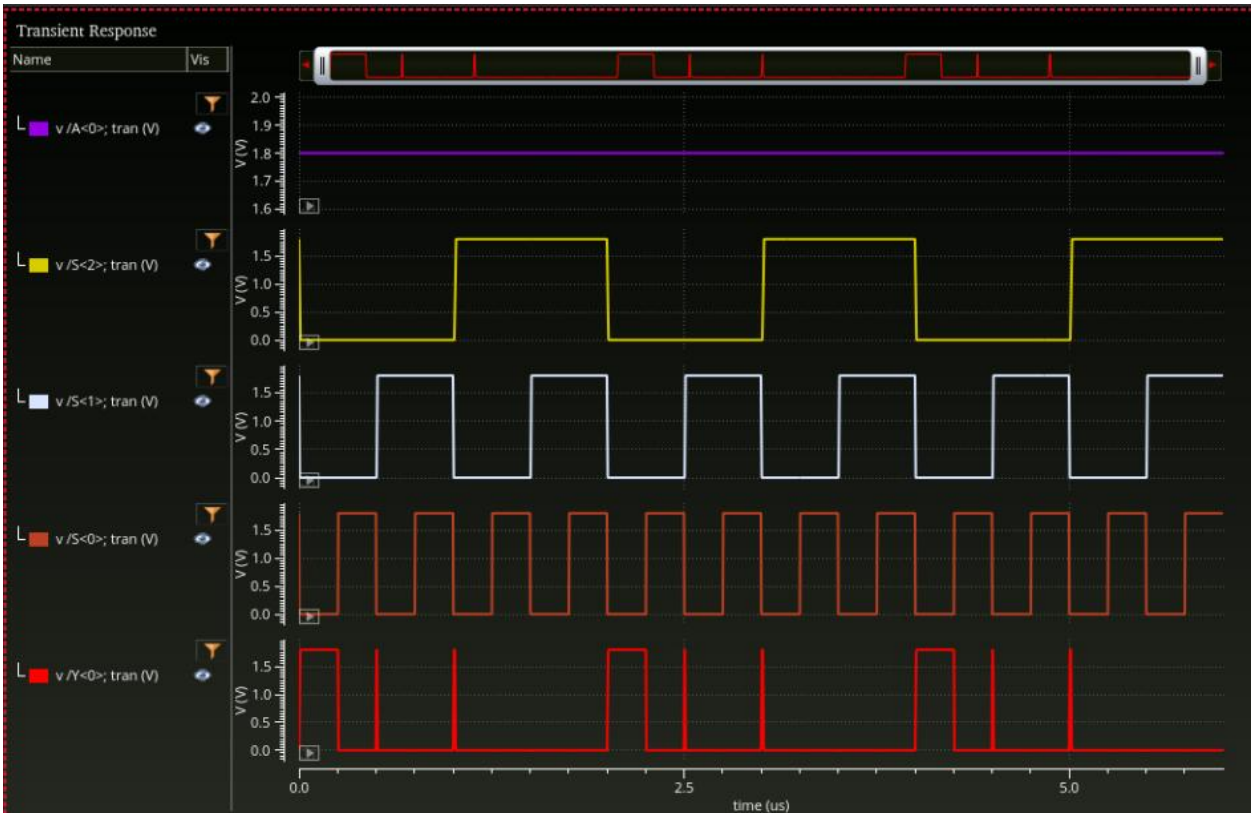

Figure 33: Correct functionality to pass only A (S<3:0> = 000)

2-Input AND

Transistor Level Schematic

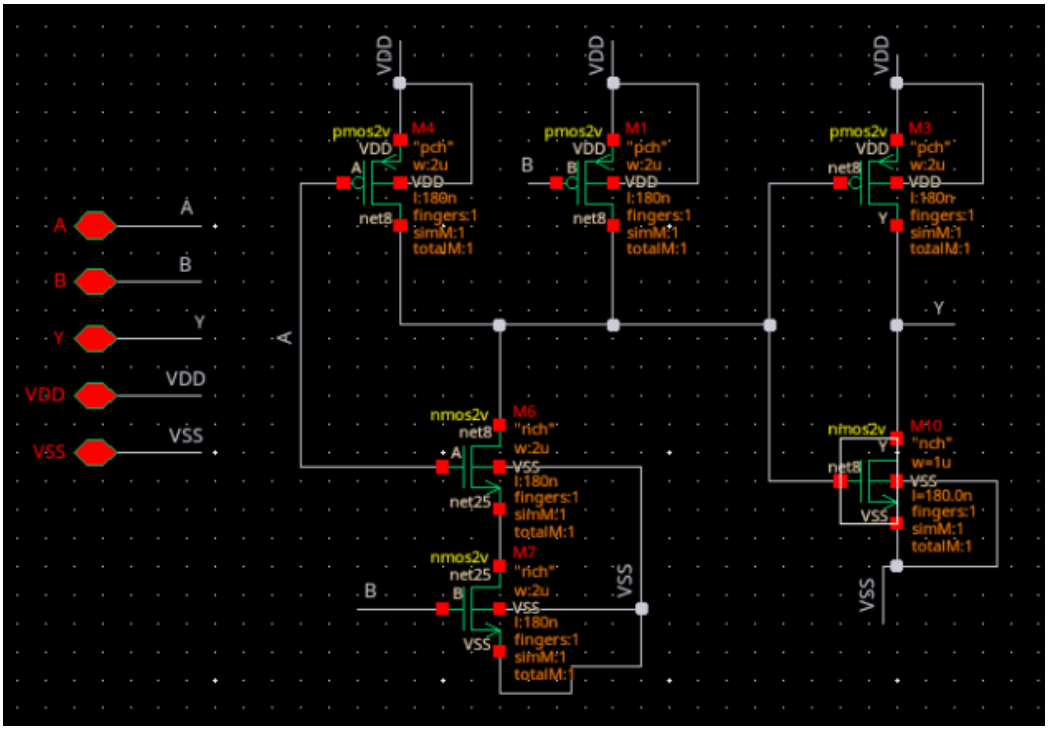

Figure 34: 2-input AND Transistor Level Schematic

Symbol

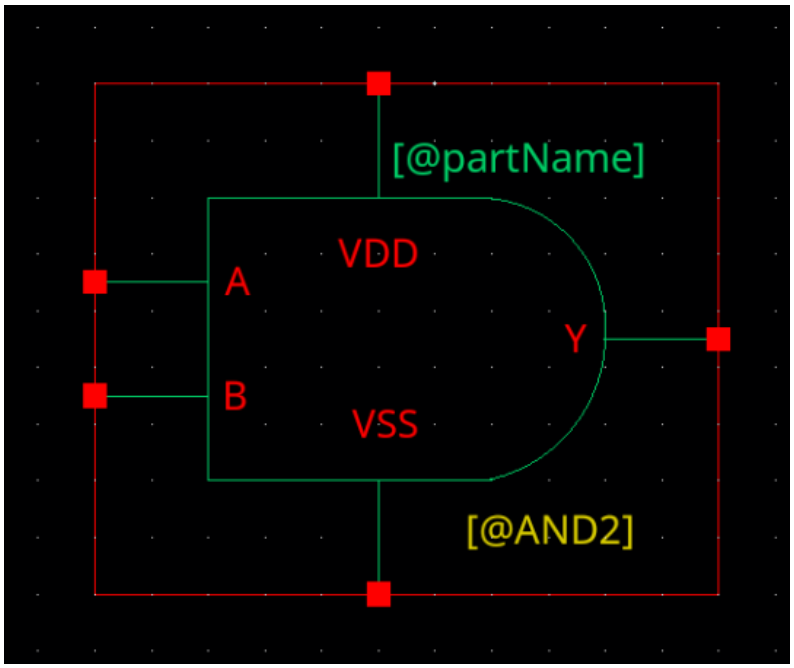

Figure 35: 2-input AND Symbol

Test Bench & Test Cases

Table 7: AND Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

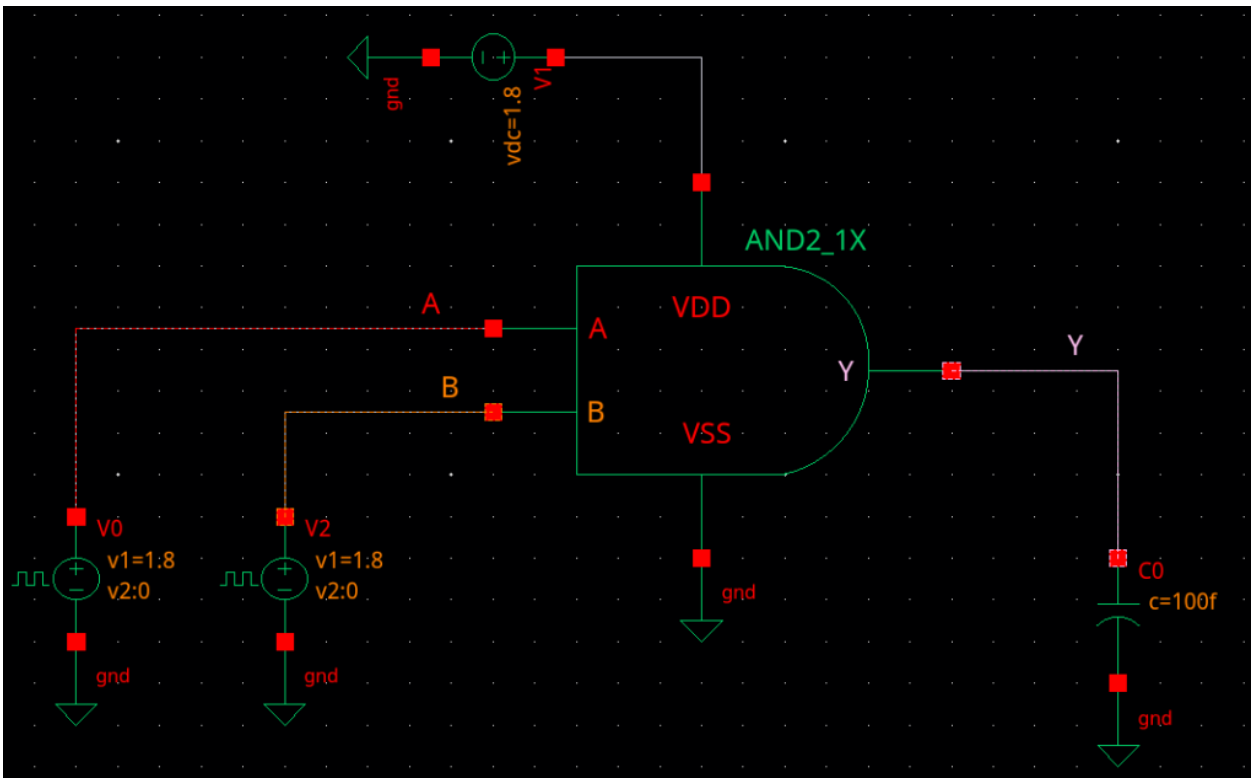

Figure 36: AND Test Bench

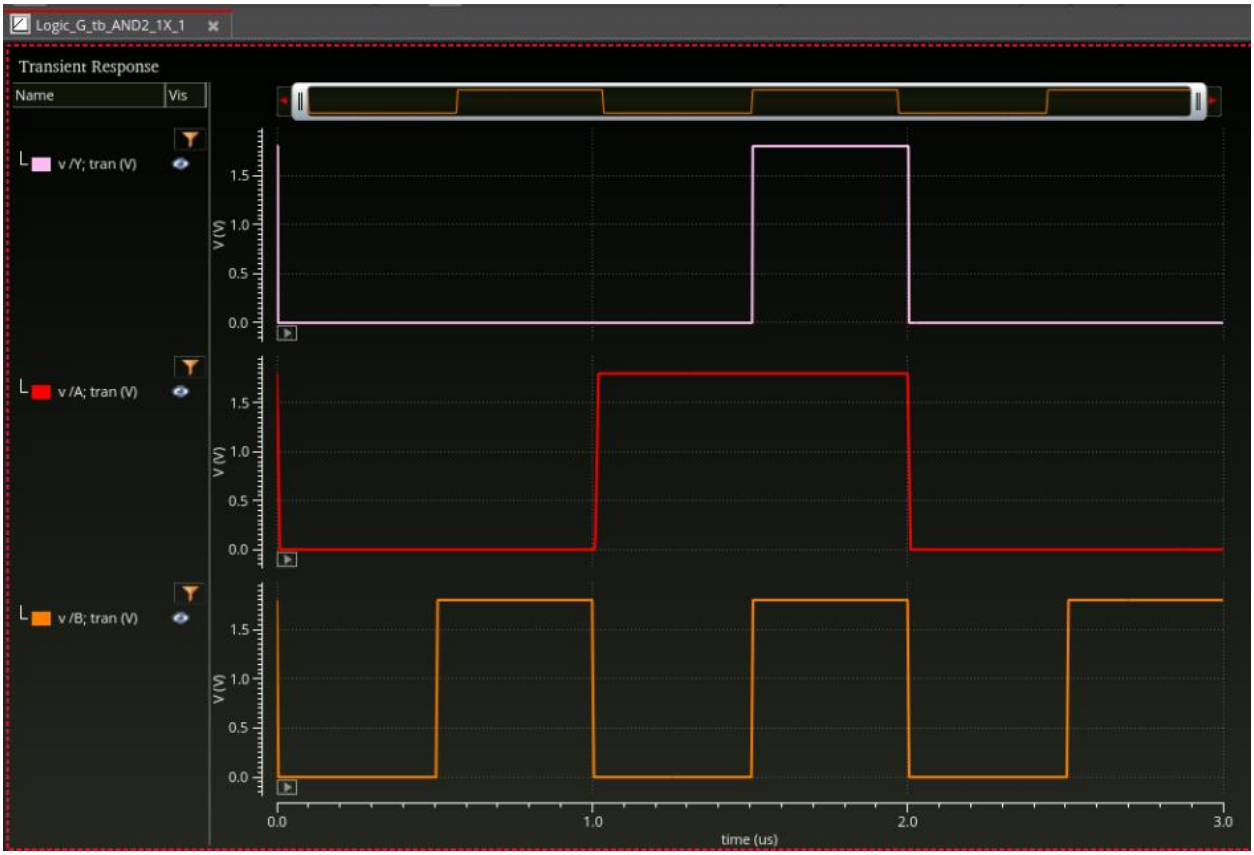

Figure 37: AND Timing Diagram

2-Input AND (4 Bits)

Block Level Schematic

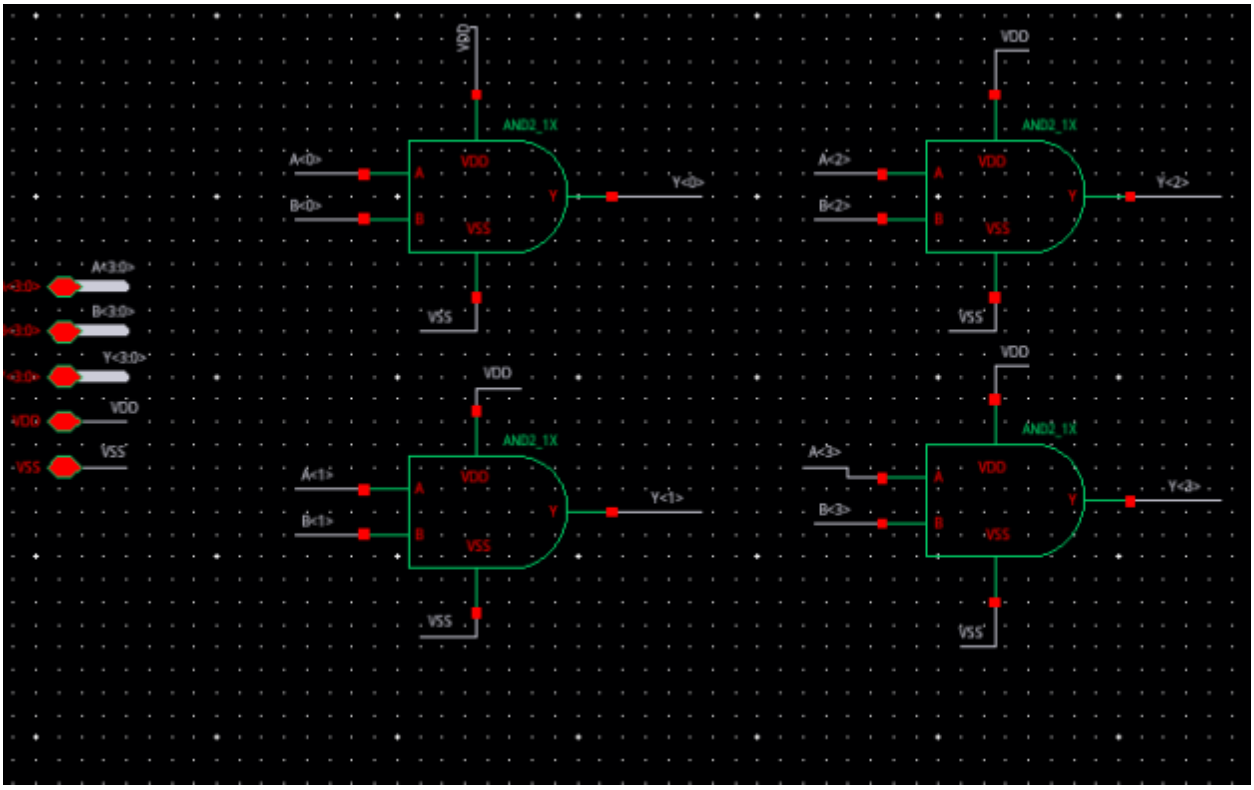

Figure 38: 4-Bit 2-Input AND Block Level Schematic

Symbol

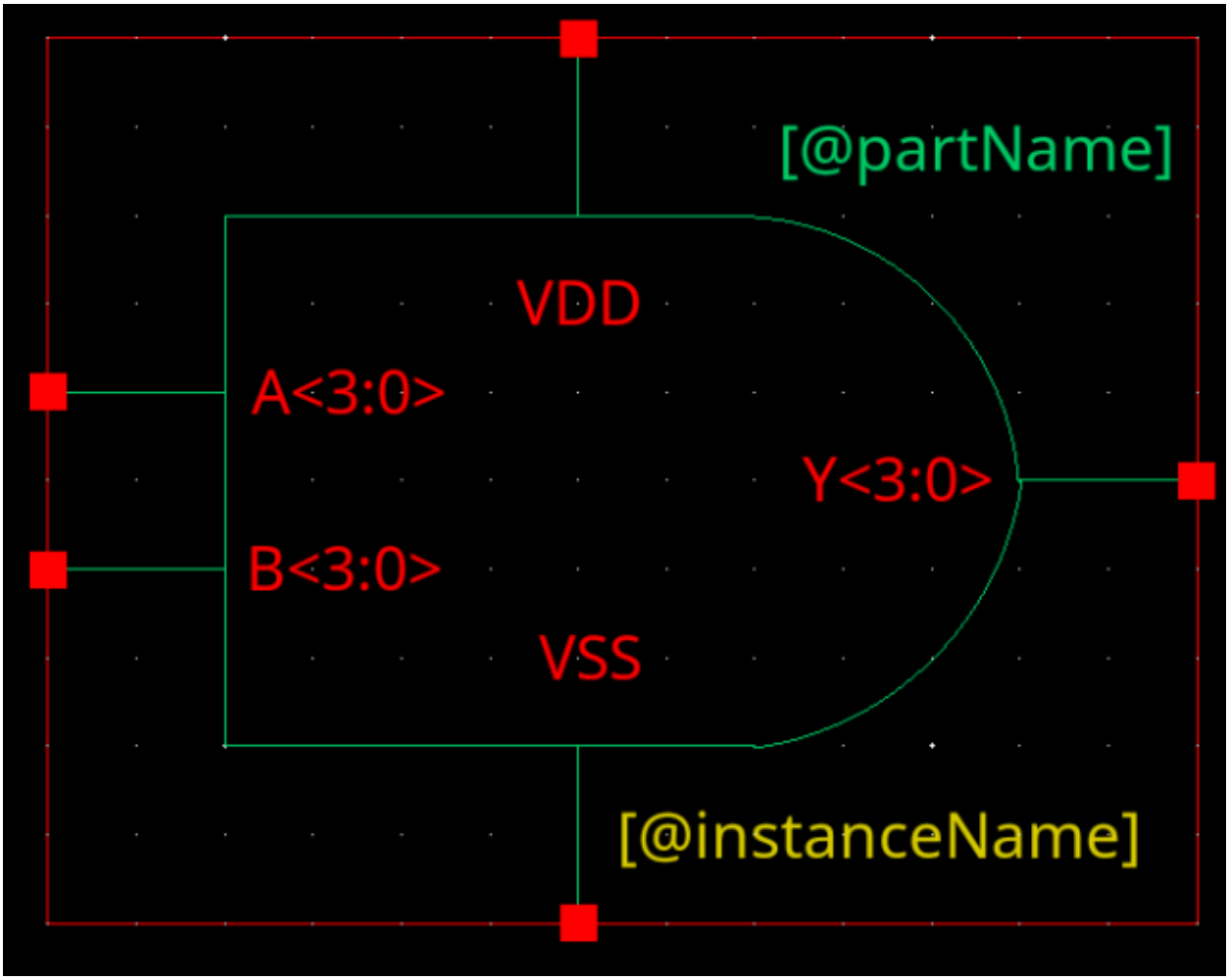

Figure 39: 4-Bit 2-Input AND Symbol

2-Input OR:

Transistor Level Schematic

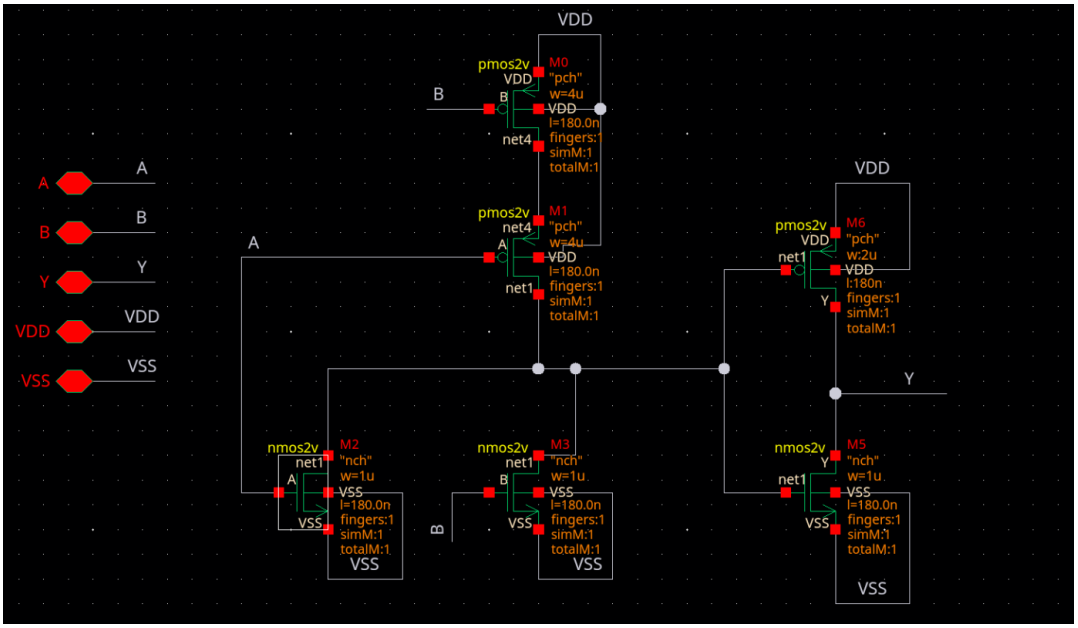

Figure 40: 2-Input OR Transistor Level Schematic

Symbol

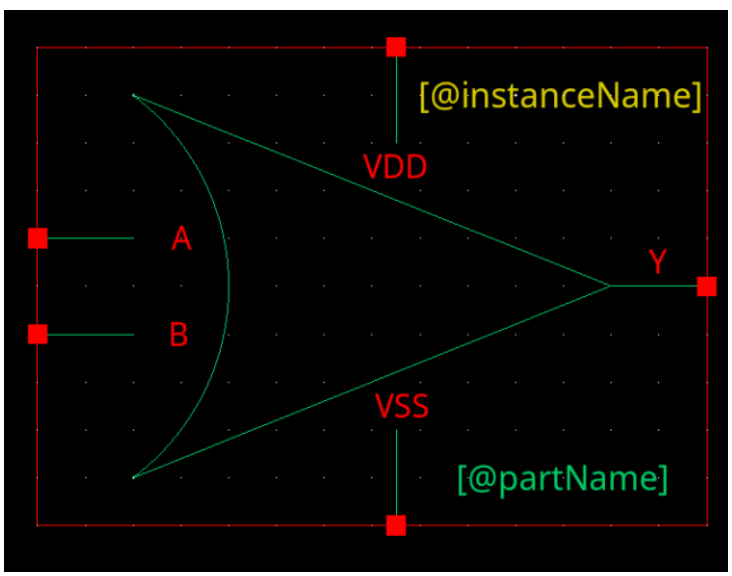

Figure 41: 2-Input OR Symbol

Test Bench & Test Cases

Table 8: 2-Input OR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

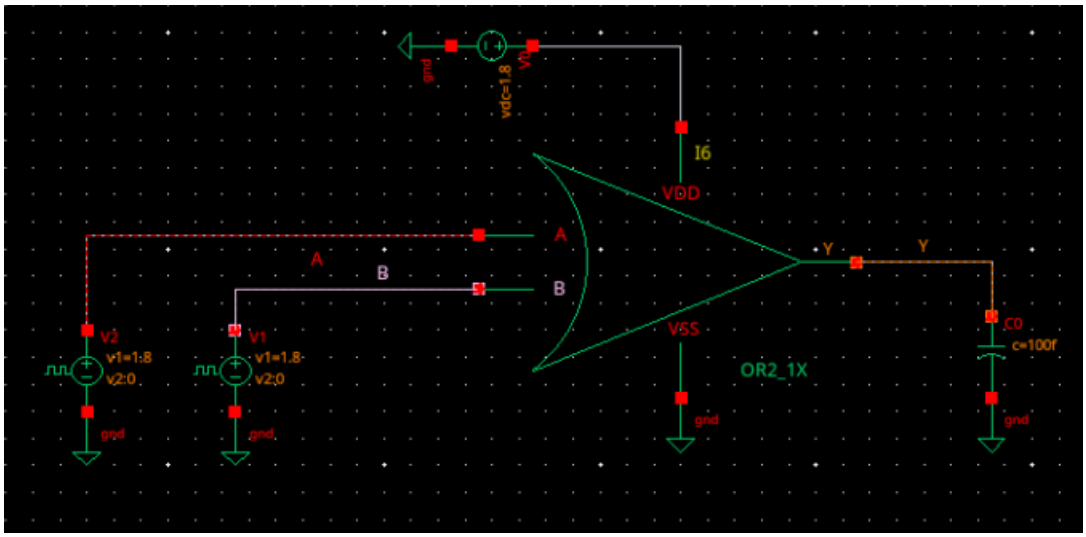

Figure 42: 2-input OR Test Bench

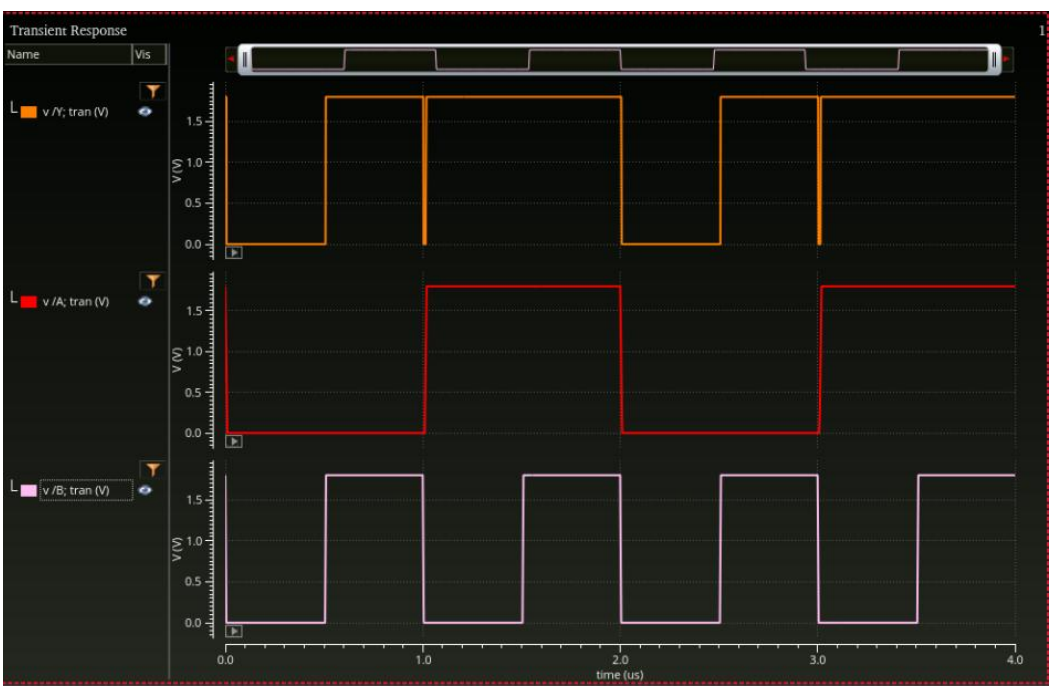

Figure 43: 2-input OR Timing Diagram

2-Input OR (4 bits)

Gate Level Schematic

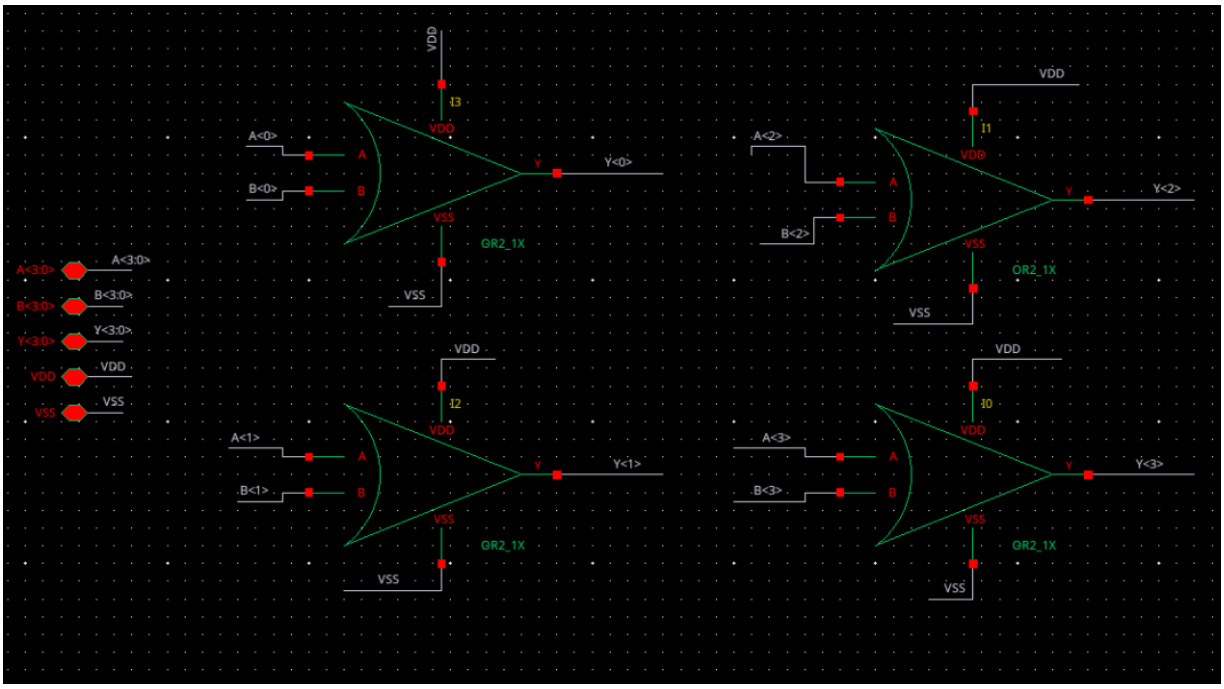

Figure 44: 4-Bit 2-Input OR Schematic

Symbol

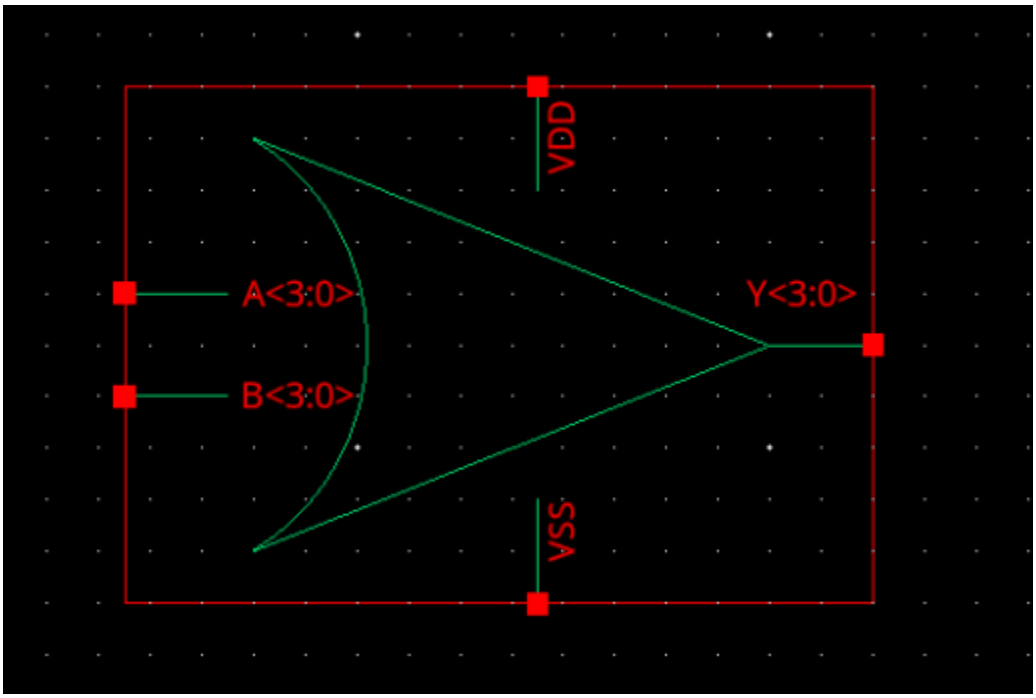

Figure 45: 4-Bit 2-Input OR Symbol

2-Input XOR

Transistor Level Schematic

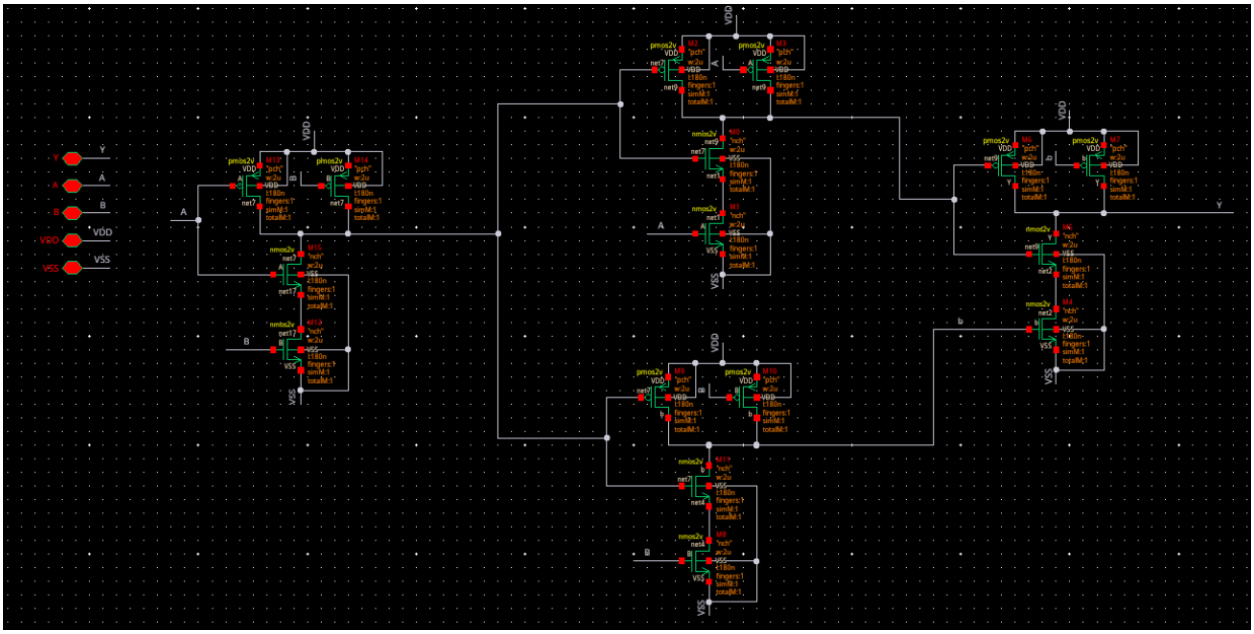

Figure 46: 2-Input XOR Transistor Level Schematic

Symbol

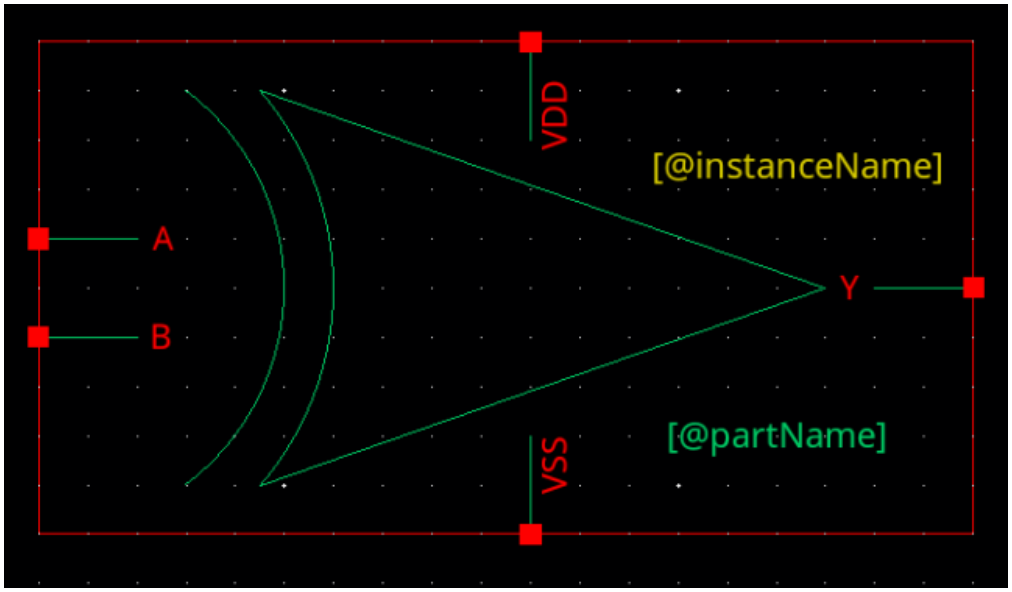

Figure 47: 2-Input XOR Symbol

Test Bench & Test Cases

Table 9: 2-Input XOR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

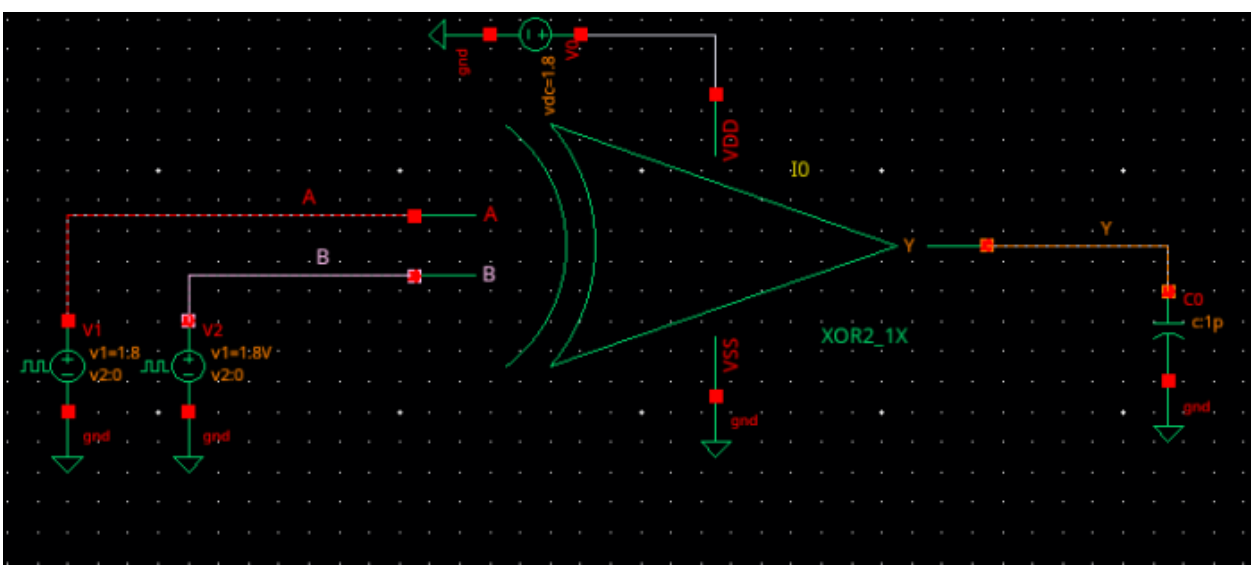

Figure 48: 2-Input XOR Test Bench

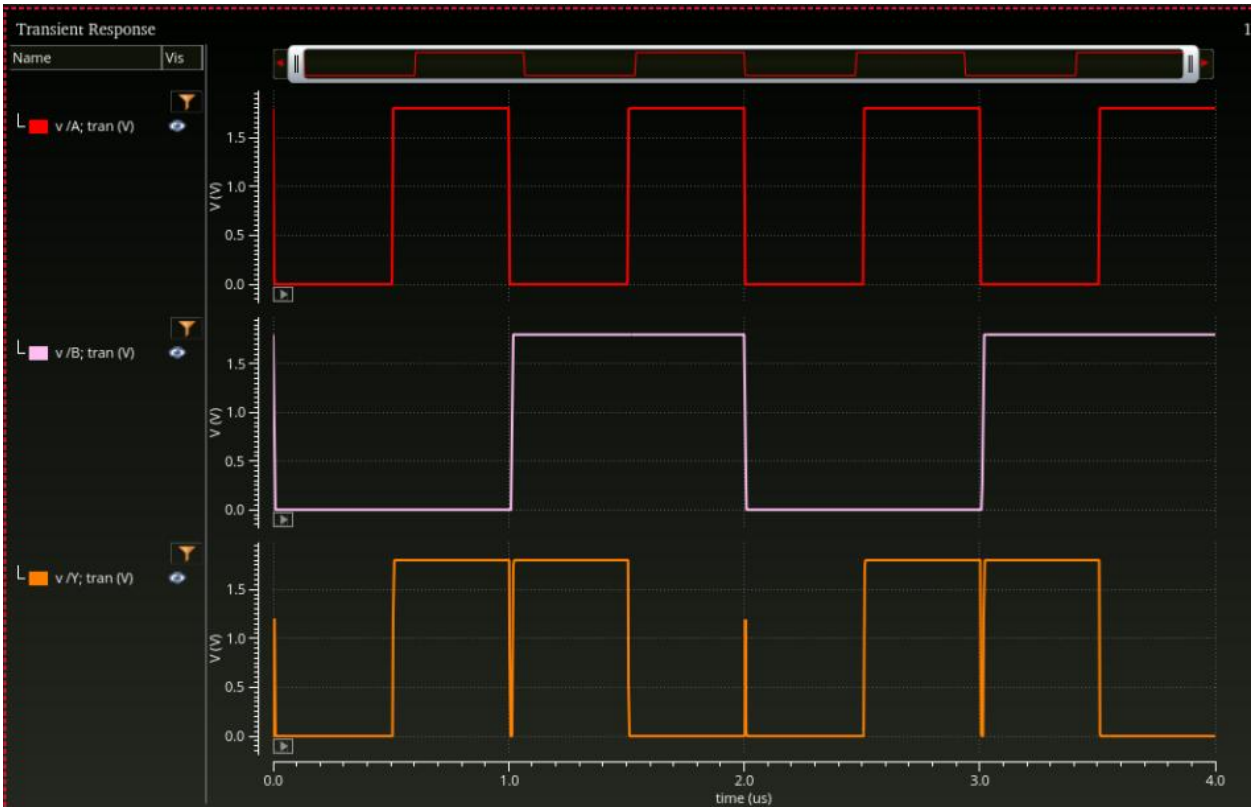

Figure 50: 2-Input XOR Timing Diagram

2-input XOR (4-Bit)

Gate Level Schematic

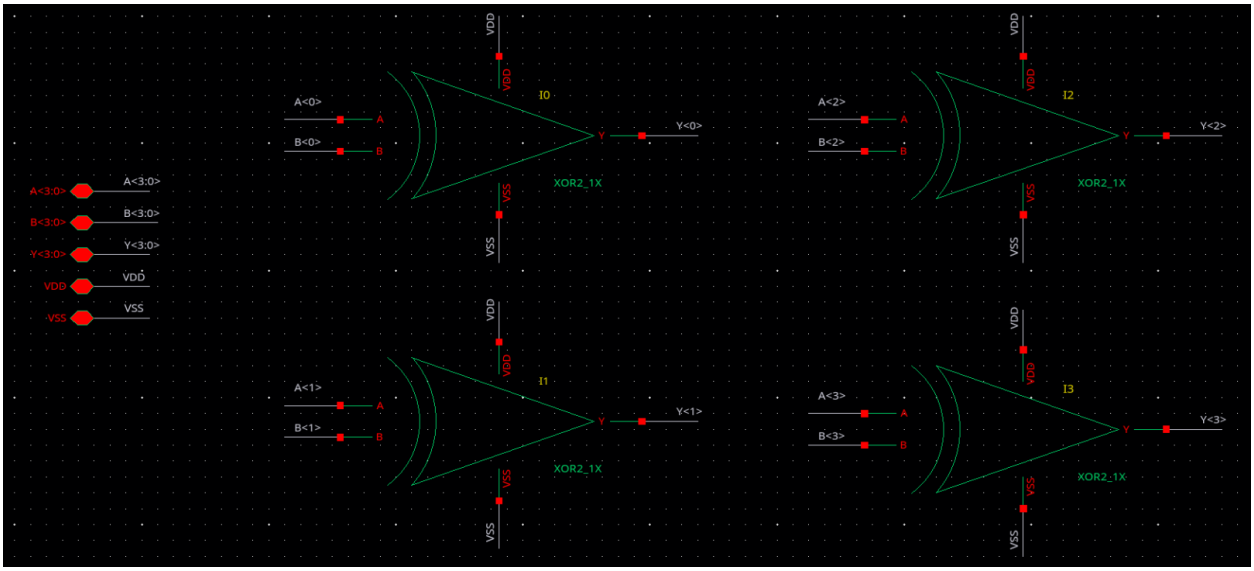

Figure 51: 4-Bit 2-Input XOR Gate Level Schematic

Symbol

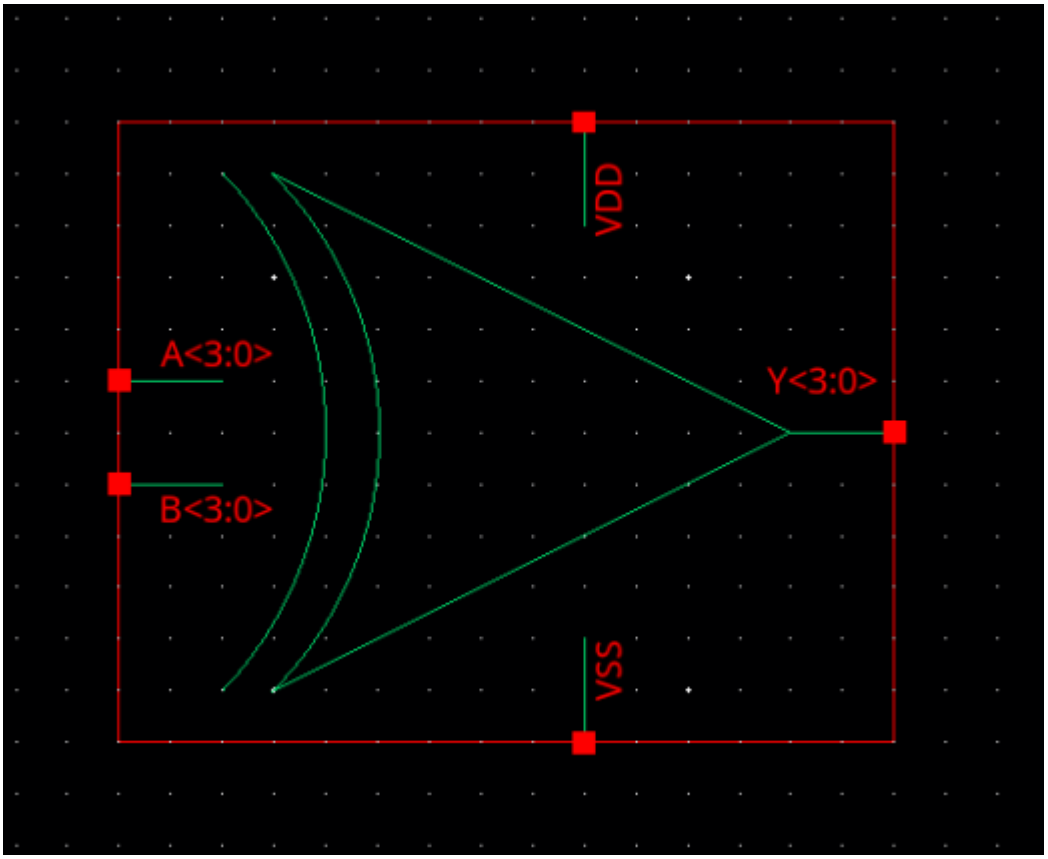

Figure 52: 4-Bit 2-Input XOR Symbol

Full Adder Layout

Schematic

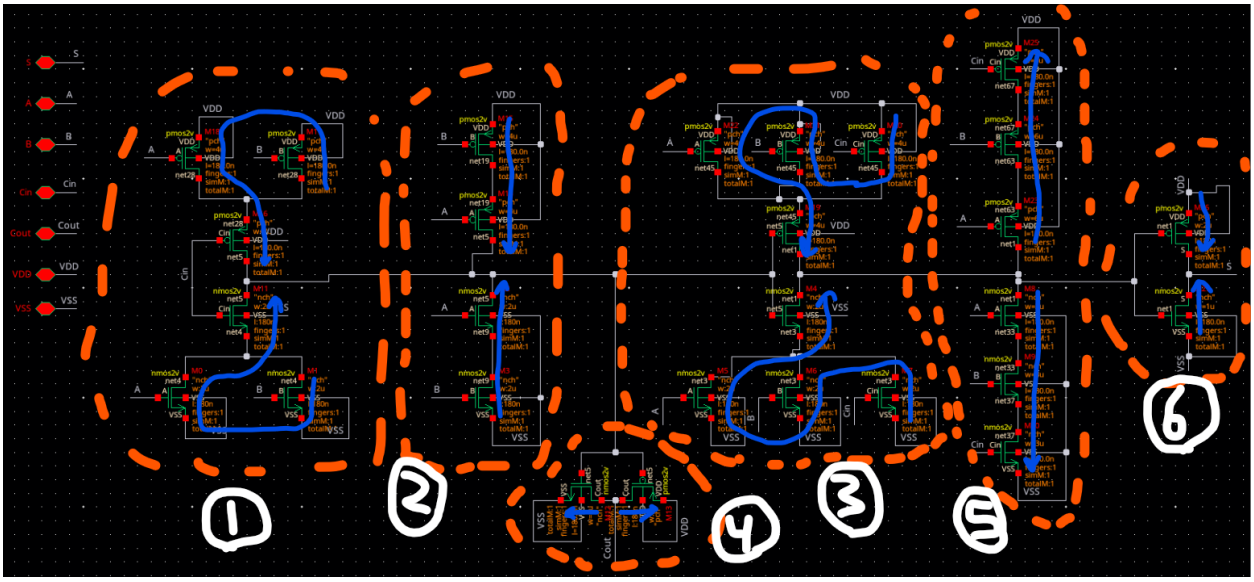

Figure 53: Schematic showing Euler Path of sections 1-6

Stick Diagram

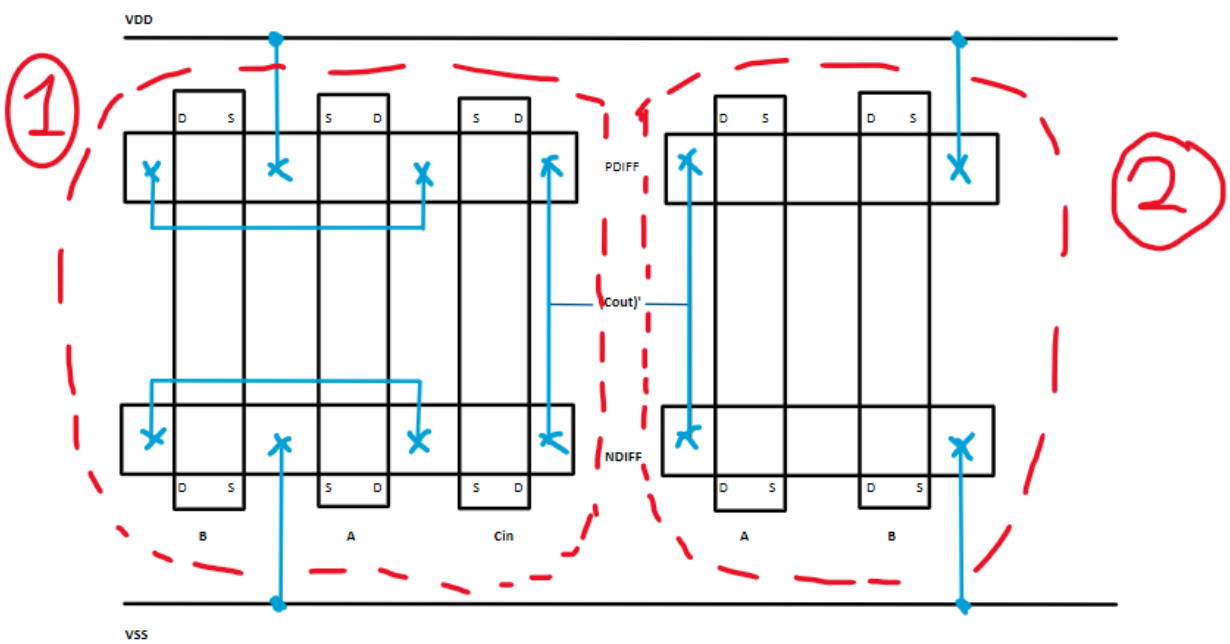

Figure 54: Stick Diagram of Sections 1 & 2

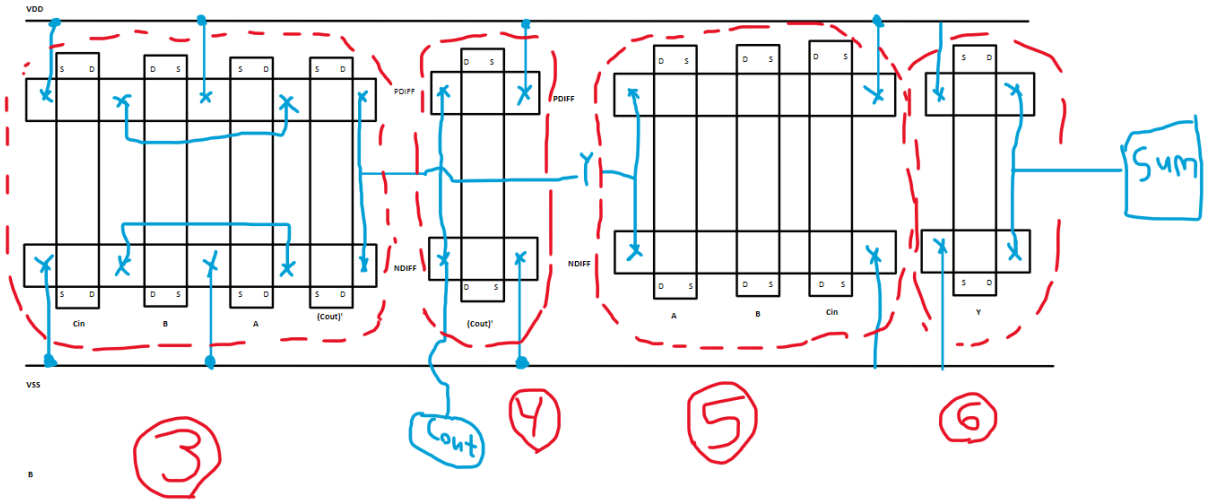

Figure 55: Stick Diagram of Sections 3 - 6

Layout

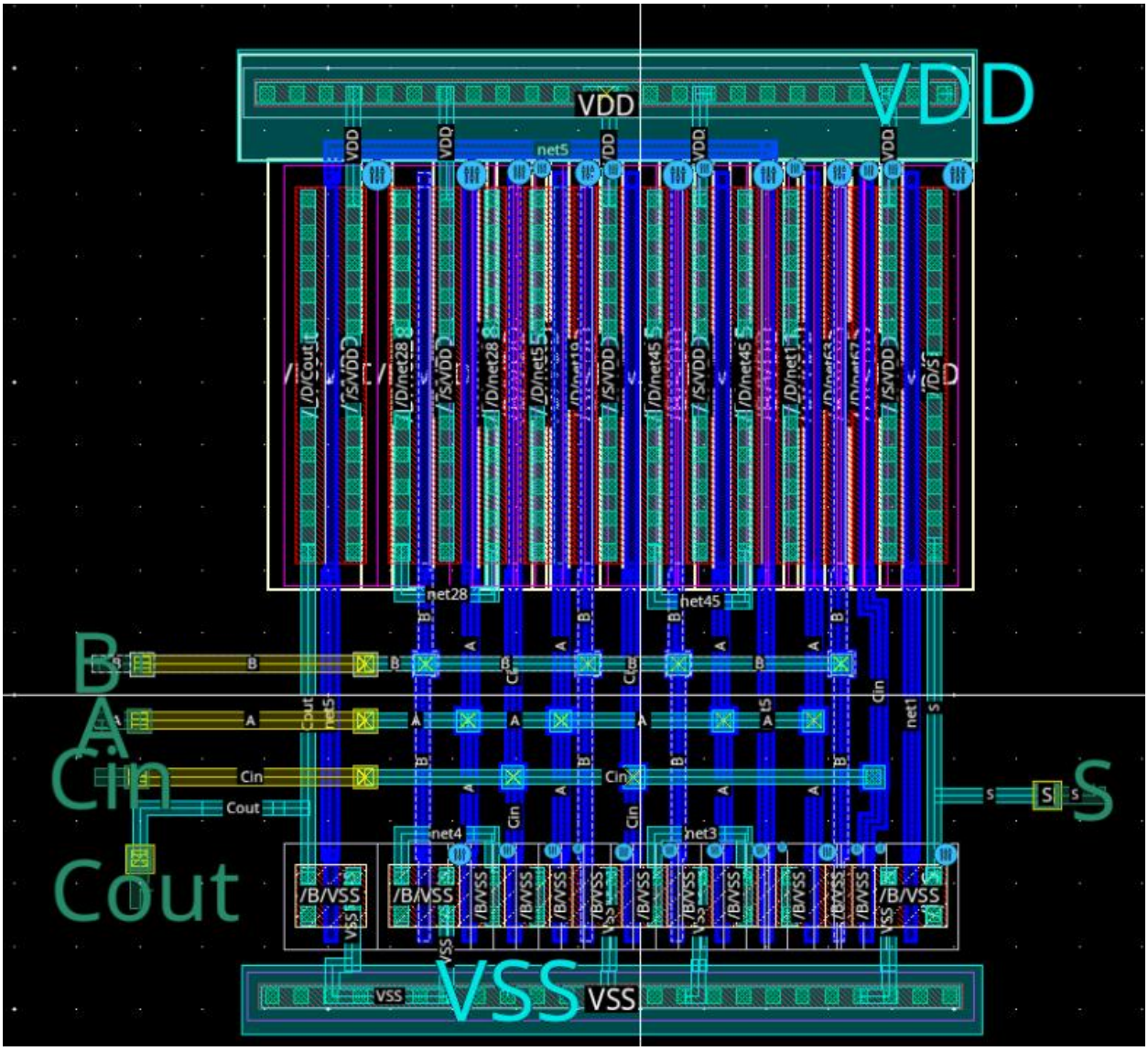

Figure 56: Full Adder Layout

DRC

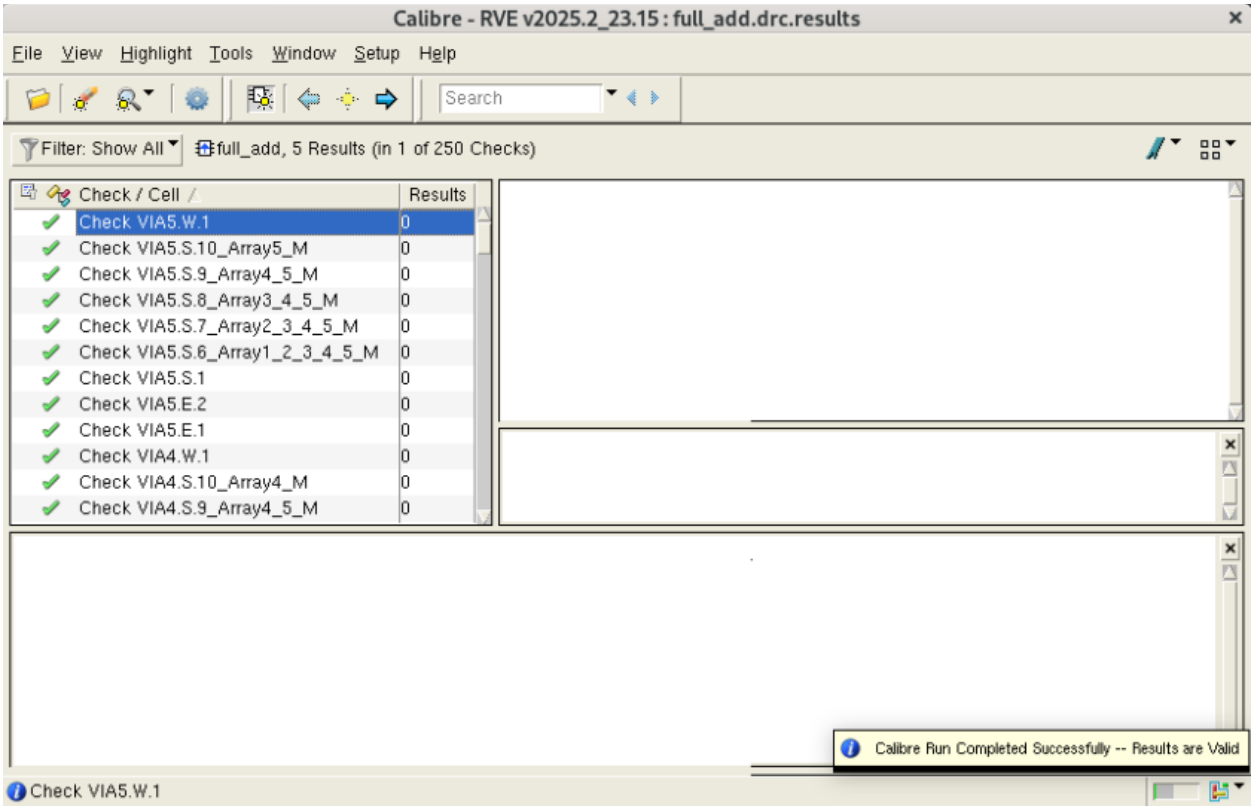

Figure 57: DRC Check

References

[1] GeeksforGeeks, “Multiplexers in Digital Logic,” *GeeksforGeeks*, Mar. 21, 2017. https://www.geeksforgeeks.org/digital-logic/multiplexers-in-digital-logic/